



Python

Install

See:

<https://docs.anaconda.com/anaconda/install/>

<https://docs.anaconda.com/anaconda/user-guide/getting-started/>

Install also:

Notebook, Spyder, Orange3

Definition

- Python is an easy to learn, powerful programming language.
- It has efficient high-level data structures and a simple but effective approach to object-oriented programming.
- Python's elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.

More Info

- Python is cross platform (for many platforms).
- The standard library is huge, very useful, and is very well documented.
 - <http://docs.python.org/library/>
- Many companies make significant use of it
 - Google uses it for a lot of their internal software projects

More Info

- Created by [Guido van Rossum](#) and first released in 1991, Python's design philosophy emphasizes [code readability](#) with its notable use of [significant whitespace](#)
- Python is [dynamically typed](#) and [garbage-collected](#). It supports multiple [programming paradigms](#), including [structured](#) (particularly, [procedural](#)), object-oriented, and [functional programming](#)

Why is it called Python?

When he began implementing Python, Guido van Rossum was also reading the published scripts from [“Monty Python’s Flying Circus”](#), a BBC comedy series from the 1970s.





2- Basic data types

Objectives



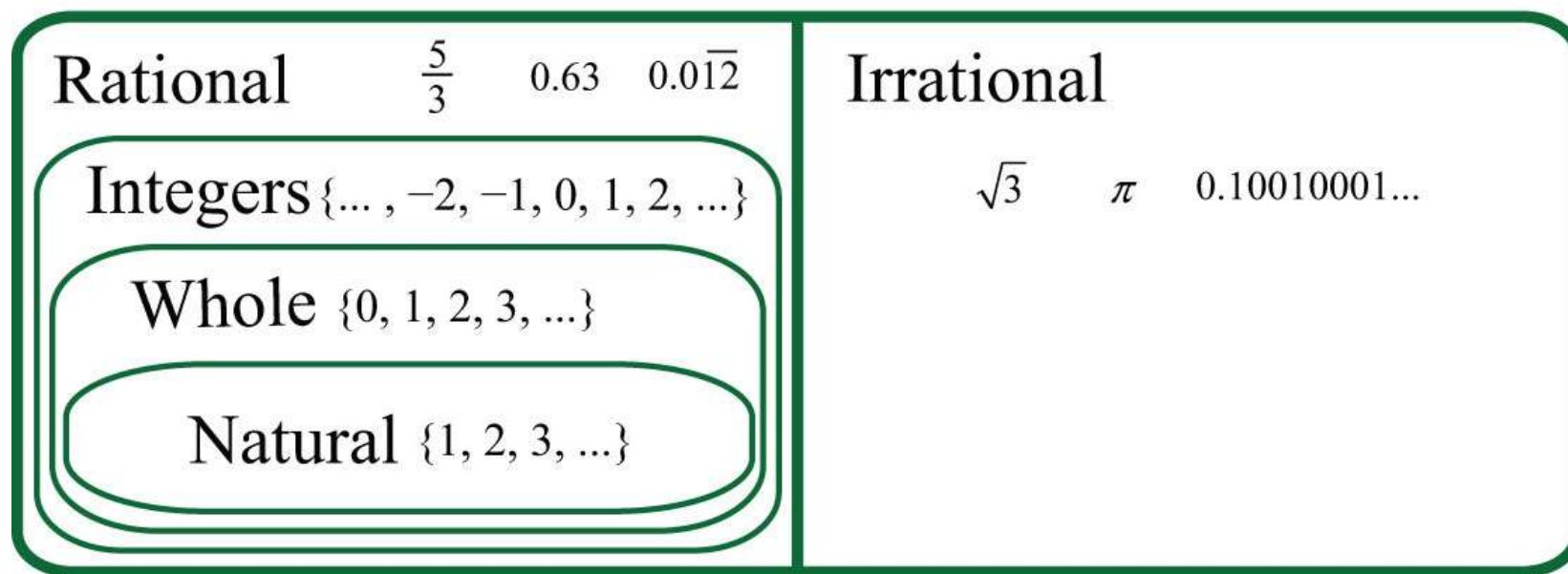
-
- Understand how numbers and characters are represented in computers
 - Understand and use operators for numbers
 - Understand arithmetic expression
 - Understand data type and use type conversion
 - To be able to read and write programs that process numerical data.

Real number system



Recall from your math class, real numbers consist of rational numbers and irrational numbers.

Real Numbers



Numeric data types



- Computers “simulate” the real number system.
- Two numeric data types:
 - Integer (`int`), e.g., 10, 0, -9999
 - Floating-point number (`float`), e.g., 1.1, 0., -3333.33
- `int` and `float` are two different *data types*.
- A floating-point number can be represented by an exponent component, e.g., -3.33333×10^3 (type -3.33333e3 in Python)
- Inside the computer, integers and floating point are represented quite differently.

Limits of range and precision

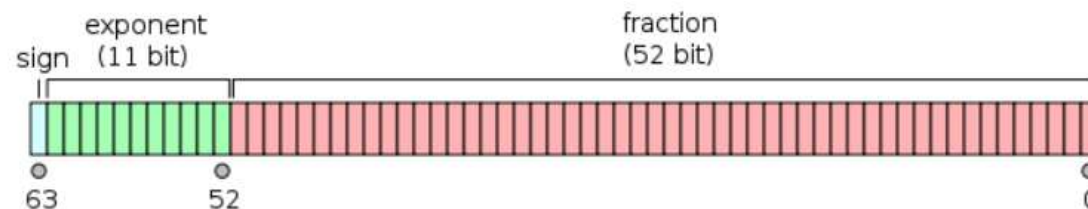


The size of an integer in Python is only limited by the memory that stores it.

Floating-point values are presented by a double-precision format (IEEE 754).

- A range of 10^{-308} to 10^{308} with 16 to 17 digits of precision

Arithmeti



The real value assumed by a given 64-bit double-precision datum with a given biased exponent e and a 52-bit fraction is

$$(-1)^{\text{sign}}(1.b_{51}b_{50}\dots b_0)_2 \times 2^{e-1023}$$

or

$$(-1)^{\text{sign}} \left(1 + \sum_{i=1}^{52} b_{52-i} 2^{-i} \right) \times 2^{e-1023}$$

Exercise 2.1



- Enter $3 * (1/3)$. Does the result match your expectation?
- Enter $1/3 + 1/3 + 1/3 + 1/3 + 1/3 + 1/3$. Does the result match your expectation?

Rounding



The displayed value has been rounded.

Several related functions:

- `round(x, n)` built-in function
- `math.ceil(x)` math function
- `math.floor(x)` math function

Exercise 2.2



- Try `round(0.45, 1)`, `round(1.45, 1)`, `round(2.45, 1)`, ..., `round(9.45, 1)`. Do you observe any patterns?
- Try `math.ceil(5.45)` and `math.floor(5.45)`.
- Try `int(5.45)` and `float(5)`.

String



- Strings in Python can be expressed inside double quotes or single quotes.
 - A string can be empty.
- Strings in Python are represented by UTF-8 using 8 to 32 bits to represent a character.
 - The 8-bit representation is the same as the ASCII.
- The `ord` function returns the numeric (ordinal) code of a single character, e.g., `ord('A')` is 65.
- The `chr` function converts a numeric code to the corresponding character, e.g., `chr(65)` is 'A'.

ASCII



Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Exercise 2.3



Try the followings:

- `print('')` // two single quotes
- `print("")` // two double quotes
- `print(" ' ")`
- `print(' " ')`
- `print("'")` // double quote + single quote
- `ord('')` // two single quotes
- `ord(' ')`

Control characters



Control characters are special characters that are not displayed on the screen, and they control the display of output (among other things).

An escape sequence begins with an escape character (\) that causes the sequence of characters following it to “escape” their normal meaning.

Escape Sequence	Meaning
<code>\newline</code>	Ignored
<code>\\</code>	Backslash (\)
<code>\'</code>	Single quote (')
<code>\"</code>	Double quote (")
<code>\a</code>	ASCII Bell (BEL)
<code>\b</code>	ASCII Backspace (BS)
<code>\f</code>	ASCII Formfeed (FF)
<code>\n</code>	ASCII Linefeed (LF)
<code>\r</code>	ASCII Carriage Return (CR)
<code>\t</code>	ASCII Horizontal Tab (TAB)
<code>\v</code>	ASCII Vertical Tab (VT)
<code>\ooo</code>	ASCII character with octal value <i>ooo</i>
<code>\xhh...</code>	ASCII character with hex value <i>hh...</i>

Exercise 2.4



Try

- `print("1\t2\t3")`
- `print("1\n2\n3")`
- `print("1\r2\r3")`
- `print("\")`
- `print("\\")`
- `print("\")`

Variables and identifiers

- Besides literals, a variable can be used to hold a value.
 - A variable is a name (identifier) that is associated with a value
- An identifier is also used to name functions and modules.
- Every identifier must begin with a letter or underscore (“_”), followed by any sequence of letters, digits, or underscores.
 - Note that “—”, “.” and many other symbols are not allowed.
 - Identifiers are case sensitive.
 - Identifiers cannot be Python’s keywords.

Python's keywords

Keywords in Python programming language

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Exercise 2.5



Say, if you suspect that a word might be a Python keyword, how would you find out without looking up the table of the keywords?

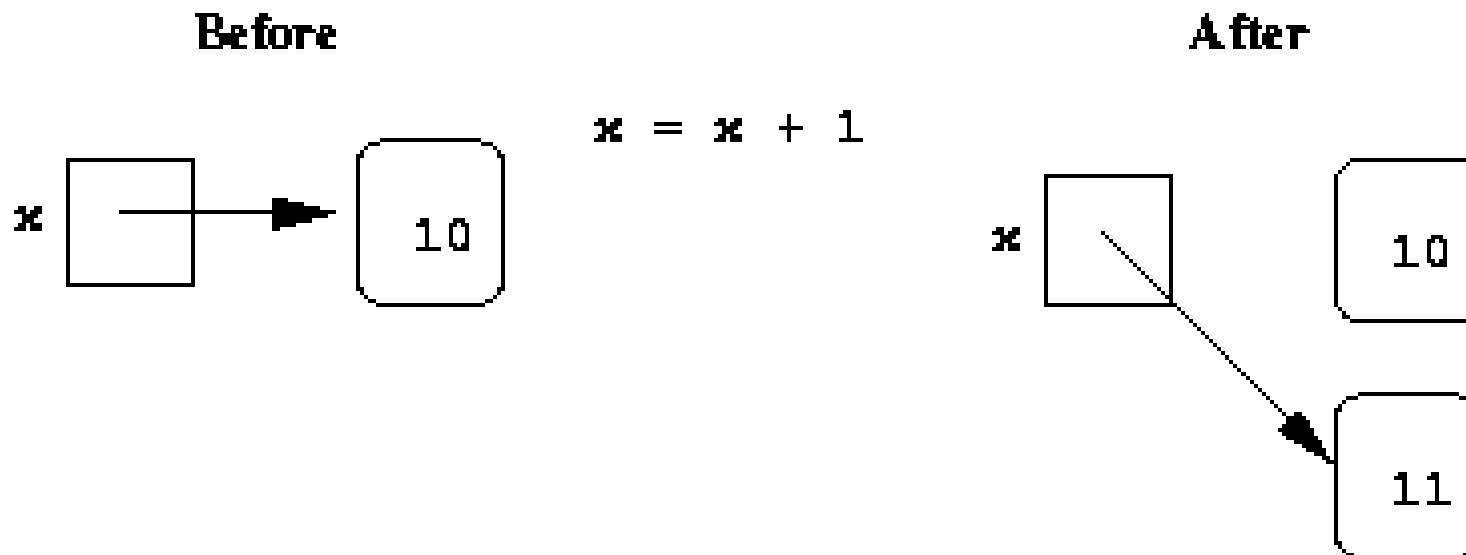
Assignment statements



- Simple assignment: `<variable> = <expr>`
variable is an identifier, expr is an expression
- The expression on the RHS is evaluated to produce a value which is then associated with the variable named on the LHS.

Behind the scene

- Python does not recycle these memory locations.
 - Integer values are immutable.
- Assigning a variable is more like putting a “sticky note” on a value and saying, “this is x”.
- The memory for the old value will be “released” automatically (i.e., *garbage collection*).



Assigning Input



- The purpose of an input statement is to get input from the user and store it into a variable.
- `<variable> = input(<prompt>)`
- E.g., `x = eval(input("Enter a temperature in Celsius: "))`
 - Print the prompt.
 - Wait for the user to enter a value and press `<enter>`.
 - The expression that was entered is evaluated and assigned to the input variable.
- Two kinds of input: character string or number

Exercise 2.6



- Prompt user for a number and then print it out using just `input (<prompt>)` .
- Prompt user for a number and then print it out using `eval (input (<prompt>))` .
- What is the difference between the two?

Exercise 2.7



Ask users to input two numbers and print out the two numbers in a reversed order.

Simultaneous Assignment



Several values can be calculated at the same time.

`<var>, <var>, ... = <expr>, <expr>, ...`

Evaluate the expressions in the RHS and assign them to the variables on the LHS.

E.g., `sum, diff = x+y, x-y`

E.g., `x, y = eval(input("Input the first
and second numbers separated by a
comma: "))`

Expressions



The fragments of code that produce or calculate new data values are called *expressions*.

A (numeric/string) *literal*, which is the simplest kind of expression, is used to represent a specific value, e.g. 10 or “rocky”.

- A simple identifier can also be an expression.

Simpler expressions can be combined using *operators* +, −, *, /, and **.

- The normal mathematical precedence applies.
- Only round parentheses can be used to change the precedence, e.g., $((x1 - x2) / 2^n) + (spam / k^{*3})$.

Try "I" + "love" + "you".

Python built-in numeric operations

Operation	Result
<code>x + y</code>	sum of <code>x</code> and <code>y</code>
<code>x - y</code>	difference of <code>x</code> and <code>y</code>
<code>x * y</code>	product of <code>x</code> and <code>y</code>
<code>x / y</code>	quotient of <code>x</code> and <code>y</code>
<code>x // y</code>	floored quotient of <code>x</code> and <code>y</code>
<code>x % y</code>	remainder of <code>x / y</code>
<code>-x</code>	<code>x</code> negated
<code>+x</code>	<code>x</code> unchanged
<code>abs(x)</code>	absolute value or magnitude of <code>x</code>
<code>int(x)</code>	<code>x</code> converted to integer
<code>float(x)</code>	<code>x</code> converted to floating point
<code>complex(re, im)</code>	a complex number with real part <code>re</code> , imaginary part <code>im</code> . <code>im</code> defaults to zero.
<code>c.conjugate()</code>	conjugate of the complex number <code>c</code>
<code>divmod(x, y)</code>	the pair <code>(x // y, x % y)</code>
<code>pow(x, y)</code>	<code>x</code> to the power <code>y</code>
<code>x ** y</code>	<code>x</code> to the power <code>y</code>

Operator precedence and associativity

The unary operators `**` and `-` (negation) have higher precedence than the four operators `+`, `-`, `*`, `/`.

For operators of the same precedence, the associativity determines the order of their operations.

Operator	Associativity
<code>**</code> (exponentiation)	right-to-left
<code>-</code> (negation)	left-to-right
<code>*</code> (mult), <code>/</code> (div), <code>//</code> (truncating div), <code>%</code> (modulo)	left-to-right
<code>+</code> (addition), <code>-</code> (subtraction)	left-to-right

Exercise 2.8



Try the followings:

- $2 * 3 * 4$
- $8 + 4 - 2$
- $8 - 4 - 2$
- $8 * 4 / 3$
- $8 / 4 / 2$

Data types



- A data type is a set of values, and a set of operators that may be applied to those values.
 - Integer, floating-point numbers and string are built-in types in Python.
- An internal representation could have different values:
 - 01000001 can be interpreted as "A" or 65
- Each literal or variable is associated with a data type (`int` and `float` for now).
- A `type(x)` function returns the data type of `x` which could be a literal or variable.
- Explicit type conversion
 - Built-in functions `int(x)` and `float(x)`.

Exercise 2.9



- Try out the `type()` function for both numeric and string literals and variables.
- Assign 10 to `x` and find out the type of `x`, and assign 10.0 to `x` and find out its type.

Exercise 2.10



What are the data types of the following arithmetic expressions: $2.0/3.0$, $2/3$, $2.0/3$, $2+3$, $2.0+3.0$, $2+3.0$, $2.0*3.0$, $2*3$, $2.0*3$?

Exercise 2.11



Try

- `int(11.1)`
- `int("11")`
- `int("11.1")`
- `float(11)`
- `float("11")`
- `float("11.1")`

References



A Tutorial on Data Representation: Integers, Floating-point Numbers, and Characters:

<https://www3.ntu.edu.sg/home/ehchua/programming/java/DataRepresentation.html>

Binary convertor

http://www.binaryconvert.com/result_double.html?decimal=049046050051052053053

3. Decision Structures

Objectives



- To understand the concept of Boolean expressions and the `bool` data type
- To understand the programming pattern simple/two-way/multiway decision and its implementation using a Python `if/if-else/if-elif-else` statement.
- To be able to read, write, and implement algorithms that employ decision structures, including those that employ sequences of decisions and nested decision structures.
- To understand the concepts of definite and indefinite loops as they are realized in the Python `while` statements.
- To understand the programming patterns interactive loop and sentinel loop and their implementations using a Python `while` statement.

Boolean Expressions (Condition



The **Boolean data type** contains two Boolean values, denoted as `True` and `False` in Python.

A **Boolean expression** is an expression that evaluates to a Boolean value.

Need a **relational operator** to evaluate a boolean expression.

The relational operators on the next slide can be applied to any set of values that has an ordering.

- Number comparison
- **Lexicographical ordering** for string comparison

Relational Operators



Python	Mathematics	Meaning
<	<	Less than
<=	$\leq$	Less than or equal to
==	=	Equal to
>=	$\geq$	Greater than or equal to
>	>	Greater than
!=	$\neq$	Not equal to

Exercise 3.1



Try

```
x, y, z = 1, 2, 3
```

```
x == y; x > y; x + y + z > 10
```

```
False = True; False > True; True >  
False
```

Exercise 3.2



Try

```
x, y, z = "Benjamin", "Ben", "ben"  
x > y; y > z; y + z > x  
"abc" > "123"; "abc" > ">>>"
```

Two other membership operators



The `in` operator is used to determine if a specific value is in a given list, returning `True` if found, and `False` otherwise.

The `not in` operator returns the opposite result.

Try

- `10 in [10, 20, 30, 40]`
- `10 not in [10, 20, 30, 40]`
- `"blue" in ["red", "yellow", "black"]`
- `"blue" not in ["red", "yellow", "black"]`
- `"o" in "peter paul and mary"`

Boolean Operators



x	y		x and y	x or y	not x
False	False		False	False	True
True	False		False	True	False
False	True		False	True	
True	True		True	True	

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Exercise 3.3



Try

- `True and False; True or False`
- `not (True) and False; not (True and False)`
- `(10 < 0) and (10 > 2); (10 < 0) or (10 > 2)`
- `not (10 < 0) or (10 > 2); not (10 < 0 or 10 > 2)`

Operator Precedence



Operator	Associativity
<code>**</code> (exponentiation)	right-to-left
<code>-</code> (negation)	left-to-right
<code>*</code> (mult), <code>/</code> (div), <code>//</code> (truncating div), <code>%</code> (modulo)	left-to-right
<code>+</code> (addition), <code>-</code> (subtraction)	left-to-right
<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>!=</code> , <code>==</code> (relational operators)	left-to-right
<code>not</code>	left-to-right
<code>and</code>	left-to-right
<code>or</code>	left-to-right

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Exercise 3.4



Try to figure out the answers before running them:

- `True or False and True and True`
- `True or not False and True and not True`
- `((True or not False) and True) and not True`

Control structures



Sequential control vs selection control vs iterative control

Calling function is still a sequential control.

A **control statement** is a statement that determines the control flow of a set of instructions.

A **control structure** is a set of instructions and the control statements controlling their execution.

- Three fundamental forms of control in programming are sequential, selection, and iterative control.

Simple decisions



For example,

```
if n < 1:
    print("Your input number is too low.")
if n > 10000:
    print("Your input number is too large.")
```

How about?

```
n = 10001
if (1 <= n) <= 10000:
    print("Your input number is ", str(n)+".")
```

What is the result of `1 <= 10001`?

What is the result of `True <= 10000`?

What is the overall result of `(1 <= n) <= 10000`?

To do or not to do

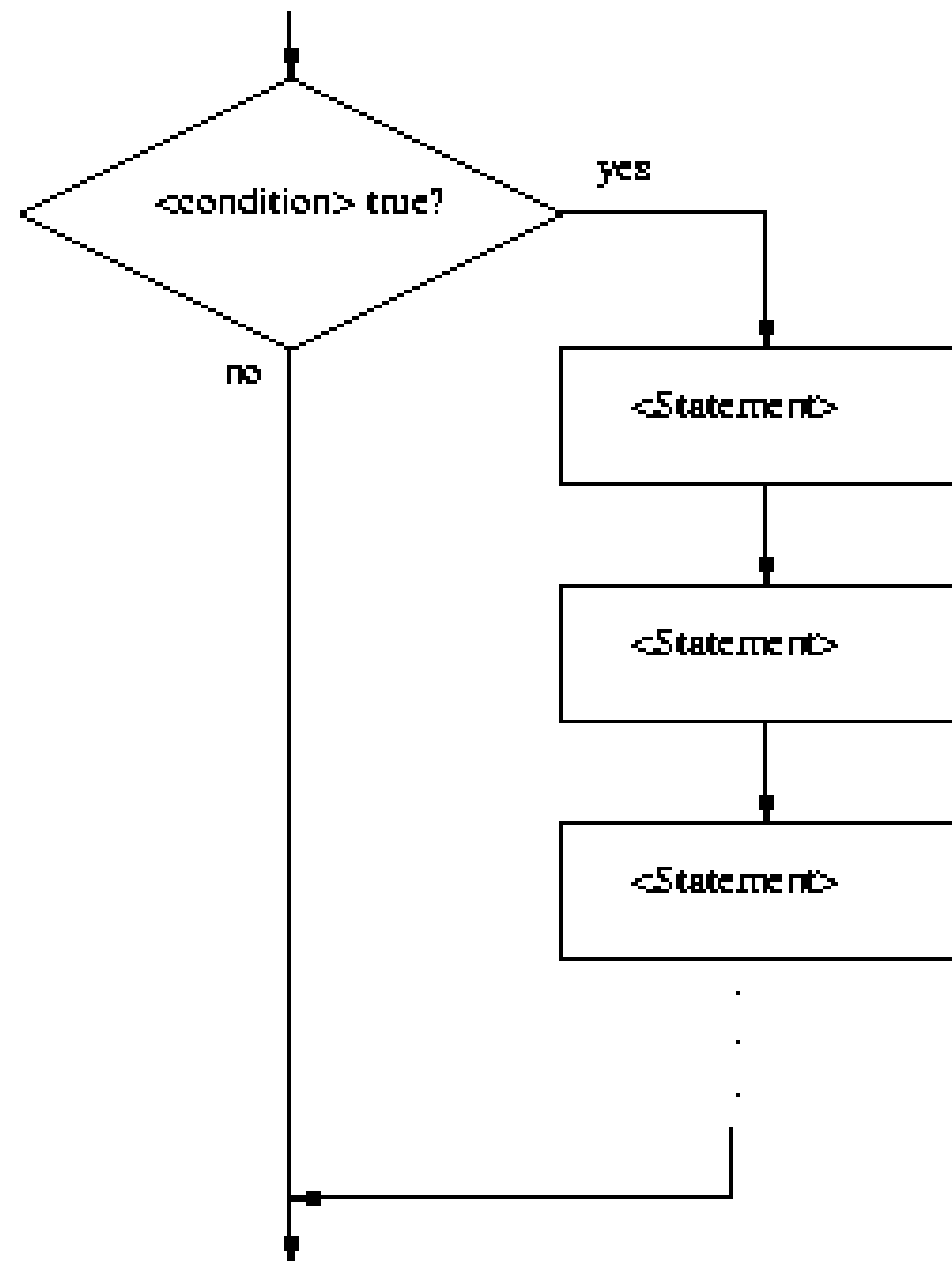
The Python `if` statement is used to implement the decision.

```
if <condition>:  
    <body>
```

The body is a sequence of one or more statements indented under the `if` heading.

The body is executed if `condition` is evaluated to `True`.

The body is skipped if `condition` is evaluated to `False`.



Two-Way Decisions



- In Python, a two-way decision can be implemented by attaching an `else` clause onto an `if` clause.

- This is called an `if-else` statement:

```
if <condition>:  
    <statements>  
else:  
    <statements>
```

- E.g.,

```
if 1 <= n <= 10000:  
    print("Your input number is ", str(n)+".")  
else:  
    print("Your input must be between 1 and 1000.")
```

Exercise 3.5



Write a program that will ask a user for an integer. It will determine whether it is odd or even by printing out a corresponding statement.

Multi-Way Decisions



- Use nested `if-else` statement to implement multi-way decisions.
- `if <condition1>:`
 `<case1 statements>`
- `else:`
 `if <condition2>:`
 `<case2 statements>`
 `else:`
 `if <condition3>:`
 `<case3 statements>`
 `else:`
 `<default statements>`

Multi-Way Decisions using `elif`

- In Python, `else-if` is combined into `elif`:

```
if <condition1>:  
    <case1 statements>  
elif <condition2>:  
    <case2 statements>  
elif <condition3>:  
    <case3 statements>  
else:  
    <default statements>
```

The `else` is optional. If there is no `else`, it is possible that no indented block would be executed.

Exercise 3.6



Write a program to ask for a user's IQ and use `if-elif` statements to print out the IQ classification according to the table on the next slide.



IQ Range ("deviation IQ")	IQ Classification ^{[32][33]}
130 and above	Very Superior
120–129	Superior
110–119	High Average
90–109	Average
80–89	Low Average
70–79	Borderline
69 and below	Extremely Low

source: http://en.wikipedia.org/wiki/IQ_classification

Exercise 3.7



Ask a user for the number of hours he has worked in a week. Hours worked over 40 are overtime, paid at 1.5 times the normal rate. The normal rate is \$32.5 per hour. Your program will calculate and print out his wage for the week.

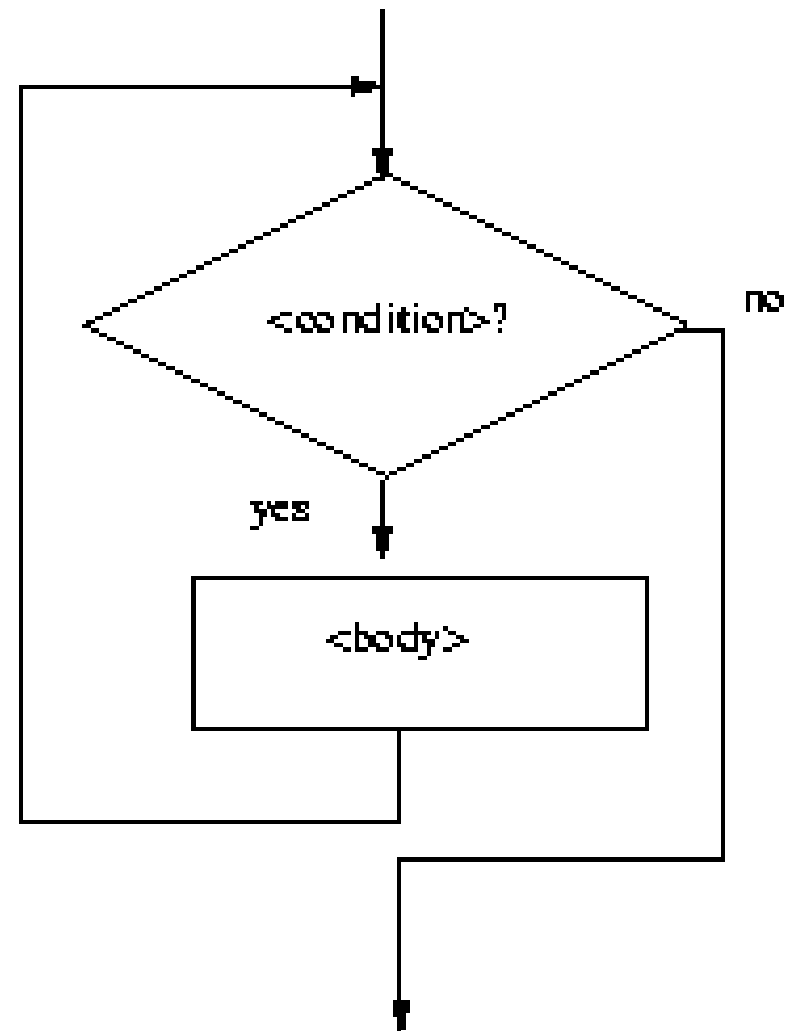
while statement

```
while <condition>:  
    <body>
```

`condition` is a Boolean expression, just like in `if` statements. The body is a sequence of one or more statements.

Semantically, the body of the loop executes repeatedly as long as the condition remains true. When the condition is false, the loop terminates.

while statement



Exercise 3.8



Try the code below and what does it do? Print out the value of `sum`.

```
sum = 0
current = 1

n = int(input('Enter value: '))

while current <= n:
    sum = sum + current
    current = current + 1
```

Exercise 3.9



Modify the code in Exercise 3.8, so that

```
while current <= n:
```

now becomes

```
while current < n:
```

How do you modify other parts of the program to make it work?

Input error checking



The while statement is well suited for input error checking in a program.

```
# Display program welcome
print('This program will convert temperatures
(Fahrenheit/Celsius)')
print('Enter (F) to convert Fahrenheit to Celsius')
print('Enter (C) to convert Celsius to Fahrenheit')

# Get temperature to convert
f_or_c = input('Please enter selection: ')

while f_or_c != 'F' and f_or_c != 'C':
    f_or_c = input('Please enter selection again: ')

temp = int(input('Enter temperature to convert: '))
```


Exercise 3.10



Finish the remaining part of the temperature conversion program. Please look up the formulae for the conversion in the Internet.

Exercise 3.11



Extend your code in Exercise 3.10 by asking user for a range of temperatures to convert. Your program will print each temperature's conversion in a new line.

Infinite loop



An **infinite loop** is an iterative control structure that never terminates (or eventually terminates with a system error).

Create an infinite loop from the code in Exercise 3.8.

Implement a mechanism that will restrict a maximum number of loops to a threshold using `break`.

Executing `break` causes Python to immediately exit the enclosing loop.

Avoid using `break` often within loops, because the logic of a loop is hard to follow when there are multiple exits.

Exercise 3.12



Rewrite the code on slide 27 as follow:

```
while True:
    f_or_c = input('Please enter
                    selection again: ')
    ...
```

Finish the remaining code.

Definite and indefinite loops



A **definite loop** is a program loop in which the number of times the loop will iterate can be determined before the loop is executed.

An **indefinite loop** is a program loop in which the number of times that the loop will iterate cannot be determined before the loop is executed.

Boolean flag and indefinite loop



Often the condition of a given while loop is denoted by a single Boolean variable, called a **Boolean flag**.

A Boolean flag is a single Boolean variable used as the condition of a given control statement is called.

Consider Exercise 3.11 in which a user is asked for a low temperature and a high temperature. Your program should check whether it is true.

```
valid_entries = False
```

```
while not valid_entries:
```

```
    low_temp = int(input('Please enter a low temperature: '))
```

```
    high_temp = int(input('Please enter a high temperature: '))
```

```
...
```

Exercise 3.13



Complete the remaining code in the last slide.

4. Sequence data type

Objectives

To understand what list and tuple are in Python and how to use them.

Understand the differences between list and tuple.

Understand sequences in Python and how to manipulate them.

Understand the string data type and how strings are represented in the computer.

Use for statement to iterate over a sequence.

Use range type to generate a sequence of integers.

List

A **list** is a linear data structure, thus its elements have a linear ordering.

- A list of names, numbers, student records, grocery, etc

You can create a list in Python by

- `myList = [1, 2, 3, 4]`
- `myGrades = ["A+", "A", "B+", "B"]`
- `myMenu = ["Sausage", "egg", "bread", "potato"]`
- `myMix = [1, "Spam ", 4, "U"]`

A **list** in Python is a mutable, linear data structure of variable length, allowing mixed-type elements.

- A list can be empty, e.g., `empty_list = []`.

Exercise 4.1

- **Enter** `myList1 = [1, 2, 3, 4]` and `myList2 = [5, 6, 7, 8]`.
- **Try**
 - `myList1[0]`
 - `myList1[2:4]`
 - `myList1 + myList2`
 - `len(myList1)`

Accessing and modifying a list

You can use indices or methods or operators or built-in functions to manipulate a list.

- The index starts from 0.

Operation	fruit = ['banana', 'apple', 'cherry']	
Replace	fruit[2] = 'coconut'	['banana', 'apple', 'coconut']
Delete	del fruit[1]	['banana', 'cherry']
Insert	fruit.insert(2, 'pear')	['banana', 'apple', 'pear', 'cherry']
Append	fruit.append('peach')	['banana', 'apple', 'cherry', 'peach']
Sort	fruit.sort()	['apple', 'banana', 'cherry']
Reverse	fruit.reverse()	['cherry', 'banana', 'apple']

Exercise 4.2



- Create a list of ['a', 1, 2, 'b'] by `x = ['a', 1, 2, 'b']`.
- Use the index approach to change the list to ['b', 2, 1, 'a'].
- Do this again: `x = ['a', 1, 2, 'b']`.
Use `id()` to find out whether the two lists are stored in the same memory location.

Exercise 4.3



Create a list of `['a', 1, 2, 'b']` and try the `insert()`, `append()`, `sort()` and `reverse()` methods.
Do they all work?

Exercise 4.4



Try `del` for the list in exercise 4.4. Also look up two methods called `pop()` and `remove()` and use them to delete an item in the list. How are they differ?

Tuples

A **tuple** is an *immutable* linear data structure.

- The elements in a tuple cannot be modified.

You can create a tuple in Python by

- `myList = (1, 2, 3, 4)`
- `myGrades = ("A+", "A", "B+", "B")`
- `myMenu = ("Sausage", "egg",
"bread", "potato")`
- `myMix = (1, "Spam ", 4, "U")`
- `myEmptiness = ()`

Tuples of one element must include a comma following the element, e.g., `(1, )` (instead of `(1)`).

Exercise 4.5



Try:

- `x = (1, 2, 3, 4)` and then `x = (1, 2, 3, 4)` again and check whether the two tuples are stored in the same location.
- `x = (9)` and `y = (9,)` and use `type()` to find out their types.
- `x = (1, 2, 3, 4)` and `x[0] = 10`

Exercise 4.6



Try `append()`, `insert()`, `sort()`, `reverse()`, and `del` on a tuple to see how they work for tuples.

Sequences

- A **sequence** in Python is a linearly ordered set of elements accessed by an index number.
 - Lists, tuples, and strings are all sequences.
 - Strings, like tuples, are immutable.
- A string is a sequence of characters.

H	e	l	l	o		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11

Exercise 4.7



- `x = 'rocky'` and then `x = 'rocky'` again and check whether the two strings are stored in the same location.
- `x = ('rocky')` and `y = ('rocky',)` and use `type()` to find out their types.
- `x = 'rocky'` and `x[0] = 'c'`

Exercise 4.8



- Create a string `x = "Hello World!"`.
- Enter `x[0:3]`, `x[5:9]`, `x[:5]`, `x[5:]`, `x[:]` and find out what you are doing.
- Try negative indices.

String slicing

We can also access a contiguous sequence of characters, called a *substring*, through a process called *slicing*.

Slicing: `<string>[<start>:<end>]`

- `start` and `end` should both be integers.
- The slice contains the substring beginning at position `start` and runs up to **but doesn't include** the position `end`.
- The defaults are the beginning and the end.

Common operations for sequences

Operation		String s = 'hello' w = '!'	Tuple s = (1,2,3,4) w = (5,6)	List s = [1,2,3,4] w = [5,6]
Length	len(s)	5	4	4
Select	s[0]	'h'	1	1
Slice	s[1:4] s[1:]	'ell' 'ello'	(2, 3, 4) (2, 3, 4)	[2, 3, 4] [2, 3, 4]
Count	s.count('e')	1	0	0
	s.count(4)	error	1	1
Index	s.index('e')	1	--	--
	s.index(3)	--	2	2
Membership	'h' in s	True	False	False
Concatenation	s + w	'hello!'	(1, 2, 3, 4, 5, 6)	[1, 2, 3, 4, 5, 6]
Minimum Value	min(s)	'e'	1	1
Maximum Value	max(s)	'o'	4	4
Sum	sum(s)	error	10	10

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Exercise 4.9



Try the operations in the table on the last slide as many as you can.

Exercise 4.10



- Set `x = "sausage"` and `y = "egg"`.
- Try (a) `x + "and" + y`, (b) `3 * x`, and (c) `(3 * x) + (2 * y)` to find out what you are doing.
- Try `len(x + "and" + y)`.
- Try `*` for a list and tuple.

Exercise 4.11



Write a program that will ask for a user's first name and last name in lower cases and will output a username that consists of the first letter in the first name and the first seven letters in the last name.

A sample output on the next page



```
>>>
```

```
This program generates computer usernames.
```

```
Please enter your first name (all lowercase): rocky
```

```
Please enter your last name (all lowercase): chang
```

```
Your username is: rchang
```

```
>>> main()
```

```
This program generates computer usernames.
```

```
Please enter your first name (all lowercase): great
```

```
Please enter your last name (all lowercase): alexandar
```

```
Your username is: galexand
```

```
>>>
```

For statement (iterating over sequences)

- A **for statement** is an iterative control statement that iterates once for each element in a specified sequence of elements.
- The `for` statement allows us to iterate through a sequence of values.

```
for <var> in <sequence>:  
    <body>
```

- The loop index variable `var` takes on each successive value in the sequence, and the statements in the body of the loop are executed once for each value.

Exercise 4.12

Try

```
for k in [1,2,3,4,5]:  
    print(k)  
for k in (1,2,3,4,5):  
    print(k)  
for ch in 'Sausage':  
    print(ch, end=" ")  
for k in ['a', 'b', 'c']:  
    print(k)
```

The range function

A built-in **range** function can be used for generating a sequence of integers that a for loop can iterate over.

- Rather than being a function, range is actually an immutable sequence type.

Usage: `range(start, stop[, step])`

- The arguments must be integers.
- If the *step* argument is omitted, it defaults to 1.
- If the *start* argument is omitted, it defaults to 0.
- Try `range(1, 10)`, `range(-1, 5)`, `range(-1, -10, -1)`.

Actually, range does not create a sequence of integers.

- It creates a *generator function* able to produce each next item of the sequence when needed.
- This saves memory, especially for long lists.

Exercise 4.13



Try

```
list(range(1, 10))
```

```
list(range(-1, 5))
```

```
list(range(-1, -10, -1))
```

```
list(range(0))
```

```
list(range(1, 0))
```

What does the built-in function `list()` do?

Iterating over sequence indices



- Consider two ways of iterating a sequence:

```
x = ['a', 'y', 'w', 'c', 'd']
```

```
for k in x:
```

```
    print(k, sep='', end='')
```

and

```
for k in range(len(x)):
```

```
    print(x[k], sep='', end='')
```

- What are the differences between the two approaches?

Exercise 4.14



Repeat exercise 4.11.

But this time you ask user to enter the first name and last name at the same time.

The two must be separated at least by a space.



5. Sequences data type II

Objectives



To understand the concept of objects, an active data type.

To understand how string objects and list objects works in Python.

To understand basic file processing concepts and techniques for reading and writing text files in Python.

To be able to understand and write programs that process textual information.

Native data types



So far, we have learned some native data types in Python.

These data types are “passive” in the sense that they are just data and they cannot compute.

Some problems:

- We cannot easily use these data types to model real-life objects in our problem solving.
- E.g., a circle (object) needs three `float` data, and a student record (object) needs numbers and strings.

The concept of objects



Languages that support object-oriented features provide object data types, such as strings and lists.

Example:

- `myName = "Rocky Chang"`
- `myGrades = ["A+", "A", "B+", "B"]`

The difference with the object data types?

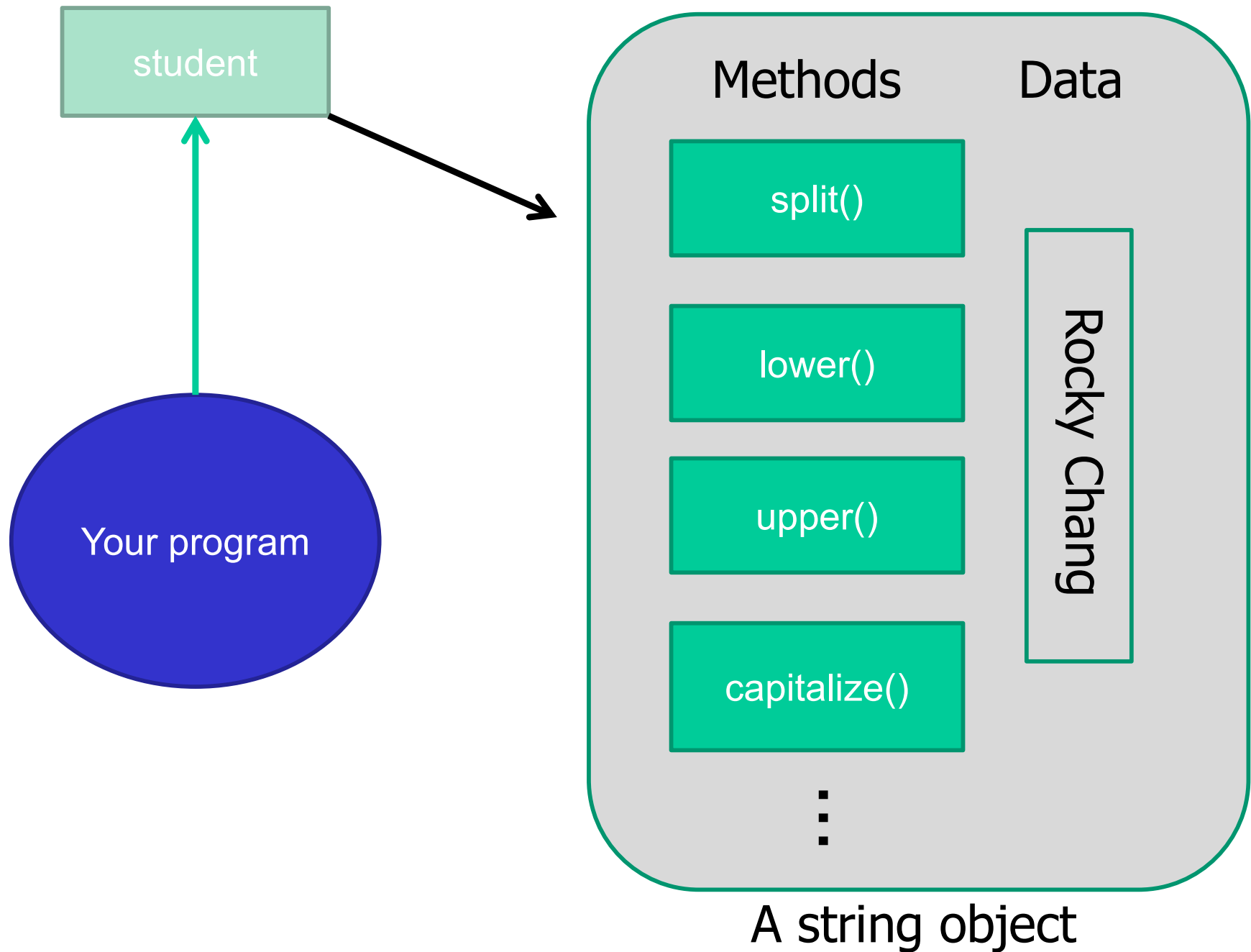
- Each object contains data (which are generally more complex).
- Each object also has methods operated in the data.

In a program, we could request an object to perform operation for us.

Exercise 5.1



- Create a string object, such as
`student = "your_first_name,
your_last_name, your student ID".`
- Invoke some methods on the object, such as
`student.split()`, `student.lower()`,
`student.upper()`, `student.capitalize()`.



String methods



For Python 3.0:

<https://docs.python.org/3/library/stdtypes.html>

- `str.capitalize()`
- `str.casefold()`
- `str.center(width[, fillchar])`
- `str.count(sub[, start[, end]])`
- `...`
- `str.title()`
- `str.translate(map)`
- `str.upper()`
- `str.zfill(width)`

Exercise 5.2



Download `decodeUnicode.py` from the course website. Fill in the missing code.

Lists are also objects.



One of the methods is `append()`.

What do these codes give us?

```
squares = []  
for x in range(1,10):  
    squares.append(x*x)  
print(squares)
```

Other methods in

<https://docs.python.org/3/library/stdtypes.html#common-sequence-operations>.

Turning a list into a string



`s.join(list)`: concatenate `list` into a string, using `s` as a separator.

- `aList = ["Rocky", "Chang"]`
- `"Memory Dennis".join(aList)`
- `"".join(aList)`

`s.join(str)`: concatenate `str` into a string, using `s` as a separator.

- `"Memory Dennis".join("Rocky Chang")`
- `"".join("Rocky Chang")`

Exercise 5.3



Create a list of "A", "B", "C", "D".
How do you use the `join()` method for
lists to return "ABCD"?

Exercise 5.4



Modify the solution to Exercise 5.2 by using a list of strings and the methods `append()` and `join()`.

Input/Output as String Manipulation



Program study: Download `dateconvert.py` from the course website for converting a date in `mm/dd/yyyy` to month, day, and year strings.

E.g., `05/24/2014` -> May 24, 2014

Pseudo-code:

- Input the date in `mm/dd/yyyy` format (`dateStr`).
- Split `dateStr` into month, day, and year strings.
- Convert the month string into a month number.
- Use the month number to lookup the month name.
- Create a new date string in the form “Month Day, Year”.
- Output the new date string.

Several things to note



Use `"/"` in `split()`.

Use `int()`, instead of `eval()`.

- `int()` can remove leading zeroes but not `eval()`.

Using `str()`



Converting a number to a string, e.g., `str(100)` → “100”.

We now have a complete set of type conversion operations:

Function	Meaning
<code>float(<expr>)</code>	Convert <code>expr</code> to a floating point value
<code>int(<expr>)</code>	Convert <code>expr</code> to an integer value
<code>long(<expr>)</code>	Convert <code>expr</code> to a long integer value
<code>str(<expr>)</code>	Return a string representation of <code>expr</code>
<code>eval(<string>)</code>	Evaluate string as an expression

String Formatting

We have so far used some simple string operations to format the output.

But it is difficult to format finer control with them.

String object has a `format` method to perform more sophisticated formatting.

The string `format()` method

String formatting syntax:

- `<template-string>.format(<values>)`
- E.g., `"The value of pi is {0:0.4f}".format(math.pi)`

Curly braces(`{ }`) inside the `template-string` marks “slots” into which the provided values are inserted.

- `{<index>:<format-specifier>}`, e.g., `{0:0.4f}`
- `index`: indicate which parameter to insert here
- One form of specifiers is `<width>.<precision><type>.`
 - `width`: the number of spaces for printing the parameter
 - `precision`: the number of decimal places
 - `f`: will print out 0 to fill up the number of decimal places.

Exercise 5.5



Try

- **Assign** `rocky = 100, dennis = 200, memory = 300.`
- `print("Rocky has ${0:0.2f} and Dennis has
${1:0.2f} and Memory has
${2:0.2f}".format(rocky, dennis, memory))`
- `print("Rocky has ${2:0.2f} and Dennis has
${1:0.2f} and Memory has
${0:0.2f}".format(rocky, dennis, memory))`
- `print("Rocky has ${3:0.2f} and Dennis has
${2:0.2f} and Memory has
${1:0.2f}".format(rocky, dennis, memory))`

Exercise 5.6



Try

- o `rocky = 100.`
- o `print("Rocky has ${0:0.2f}".format(rocky))`
- o `print("Rocky has ${0:1.2f}".format(rocky))`
- o `print("Rocky has ${0:5.2f}".format(rocky))`
- o `print("Rocky has ${0:10.2f}".format(rocky))`

Exercise 5.7



Try

```
o import math
o print("The value of pi is
    {0:0.4f}".format(math.pi))
o print("The value of pi is
    {0:0.20f}".format(math.pi))
o print("The value of pi is
    {0:0.20}".format(math.pi))
```

Exercise 5.8



Going back to our rounding mystery. Try

- o `print("The value of {0} is {0:0.40f}".format(0.45))`
- o `print("The value of {0} is {0:0.40f}".format(1.45))`
- o `print("The value of {0} is {0:0.40f}".format(2.45))`
- o `print("The value of {0} is {0:0.40f}".format(3.45))`

5. Functions

Objectives



To understand why programmers divide programs up into sets of cooperating functions.

To be able to define new functions in Python.

To understand the details of function calls and parameter passing in Python.

To understand scope of formal parameters and variables defined in a function.

To write programs that use functions to reduce code duplication and increase program modularity.

Functions everywhere

So far, we've seen three different types of functions:

- Our programs comprise a single function called `main()`.
 - You can put all your code developed before inside `main()`.
 - After running it once, you could call `main()`, instead of running code again.
- Built-in Python functions (e.g., `print()`)
- Functions from the standard libraries (`math.sqrt()`)

Why functions?

- Code reuse
- Easier to maintain (modularity)
- Facilitate solving a complex problem

Function definition

The part of the program that creates a function is called *a function definition*.

```
def <name> (<formal-parameters>) :  
    <body>
```

E.g.,

```
def avg(n1, n2, n3) :  
    print((n1+n2+n3)/3)
```

- where *n1, n2, n3* are *formal parameters*.

A function is called/invoked by `<name> (<actual-parameters>)`

- E.g., `avg(1, 2, 3)`, where 1, 2 and 3 are *actual parameters*.

Functions must be defined/imported before it is called.

Exercise 5.1



Try

```
def avg(n1,n2,n3):  
    return (n1+n2+n3)/3  
avg(1, 2, 3)  
and  
def instr(n):  
    print("The value of n is "+str(n)+''.')  
instr(10)
```

(Non-)Value-returning functions

The three arguments are passed to the three parameters according to their positions.

A **value-returning function** is a program routine called for its return value.

- The *return statement* of the form `return expr`, where `expr` may be any expression.

A **non-value-returning function** is called not for a returned value, but for its *side effects*.

- A **side effect** is an action other than returning a function value, such as displaying output on the screen.
- There is no return statement.

Exercise 5.2



What is the difference between

```
def avg(n1, n2, n3):  
    print((n1+n2+n3)/3)
```

and

```
def avg(n1, n2, n3):  
    return (n1+n2+n3)/3
```

when you invoke `x = avg(10, 20, 30)`?

(Non-)Value-returning functions

A fundamental difference in the way that value-returning and non-value-returning functions are called.

- A call to a value-returning function is an expression, e.g., `x = avg(10, 20, 30)`.
- When non-value-returning functions are called, however, the function call is a statement, e.g., `avg(10, 20, 30)`.

Nevertheless, every function in Python is technically a value-returning function.

- Any function that does not explicitly return a function automatically returns the special value `None`.

Exercise 5.3



- `def fun(x) : return`
- `def fun(x) : return`
- `def fun(x) :`
- `def fun(x) : x`
- `def fun(x) : a`

Call `fun(2)` after each function definition.

Exercise 5.4



Try

- `def fun(x) : return`
- `def fun(x) : return None`
- `def fun(x) : x`

After each function definition, execute `y = fun(2)` and print the value of `y`.

Exercise 5.5



Write a program that will generate a Celsius-Fahrenheit degree conversion program with the output like the one on the next page.

But you are required to design and implement several functions that will make your program more modular. For example, one function for getting F/C, one for getting the temperature, one for converting the temperature, and the last one for displaying the results.



This program will convert a range of temperatures
Enter (F) to convert Fahrenheit to Celsius
Enter (C) to convert Celsius to Fahrenheit

Enter selection: F

Enter starting temperature to convert: 65

Enter ending temperature to convert: 95

Degrees Fahrenheit	Degrees Celsius
-----------------------	--------------------

65.0	18.3
------	------

66.0	18.9
------	------

67.0	19.4
------	------

68.0	20.0
------	------

69.0	20.6
------	------

70.0	21.1
------	------

71.0	21.7
------	------

72.0	22.2
------	------

73.0	22.8
------	------

74.0	23.3
------	------

75.0	23.9
------	------

76.0	24.4
------	------

77.0	25.0
------	------

78.0	25.6
------	------

79.0	26.1
------	------

80.0	26.7
------	------

81.0	27.2
------	------

82.0	27.8
------	------

83.0	28.3
------	------

84.0	28.9
------	------

85.0	29.4
------	------

Exercise 5.6



Create `x1= [10, 20, 30, 40, 50]`.

Create `x2=[-10, -20, -30, -40, -50]`.

Calculate the difference between the maximum value and the minimum value in `x1`, and assign it to a variable.

Calculate the difference between the maximum in `x1` and the maximum in `x2`, and assign it to a variable. The difference should be a positive number. **Implement it using a single statement.**

Calling value-returning function

The last exercise shows that

- An expression may contain multiple function calls.
- A function call may contain function calls as arguments.

To return more than one value, we can do this by returning the two values as a tuple.

- Alternatively, we could list the values separated by commas on a return statement.

For example, write a function that can return both maximum and minimum values in a list.

Positional and keyword arguments

A **positional argument** is an argument that is assigned to a particular parameter based on its position in the argument list, e.g., exercise 6.1.

A **keyword argument** is an argument that is specified by parameter name.

- E.g.,

```
def printNumbers(n1, n2, n3):  
    print(n1, n2, n3)  
  
m1, m2, m3 = 10, 20, 30  
printNumbers(m1, m2, m3)  
printNumbers(n1=m2, n2=m3, n3=m1)
```

Default parameters

A **default argument** is an argument that can be optionally provided in a given function call.

When not provided, the corresponding parameter provides a default value.

For example, the followings give the same outputs.

- `print("The default print() parameters.")`
- `print("The default print() parameters.",
end="\n")`
- `print("The default print() parameters.",
sep=" ")`
- `print("The default print() parameters.",
end="\n", sep=" ")`
- `print("The default print() parameters.",
sep=" ", end="\n")`

Exercise 5.7



Write a function that will accept three parameters: min, max, step, all of which are integers. The function will print min, min+step, min+step*2, ..., up to the maximum value that is less than max. By default, step is set to 1.

Mutable vs. immutable arguments

Return values are the main way to send information from a function back to the caller.

Sometimes, we can communicate back to the caller by making changes to the function parameters.

For example,

```
def addInterest(balance, rate):  
    ...  
    amount = 1000  
    rate = 0.05  
    addInterest(amount, rate)
```


Exercise 5.8

Try whether this works.

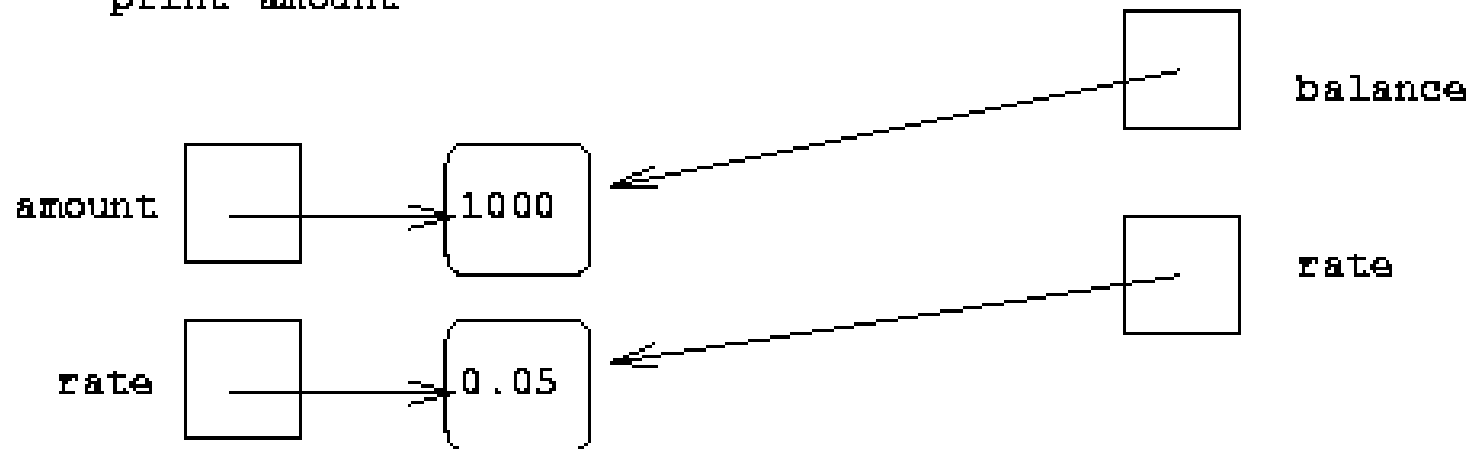
```
def addInterest(balance, rate):  
    newBalance = balance * (1 + rate)  
    balance = newBalance  
  
def test():  
    amount = 1000  
    rate = 0.05  
    addInterest(amount, rate)  
    print("My current balance is: ", amount)  
test()
```

Behind the scene

```
def test():  
    amount = 1000  
    rate = 0.05  
    addInterest(amount, rate)  
    print amount
```

balance ← amount
rate ← rate

```
def addInterest(balance, rate):  
    newBalance = balance * (1 + rate)  
    balance = newBalance
```

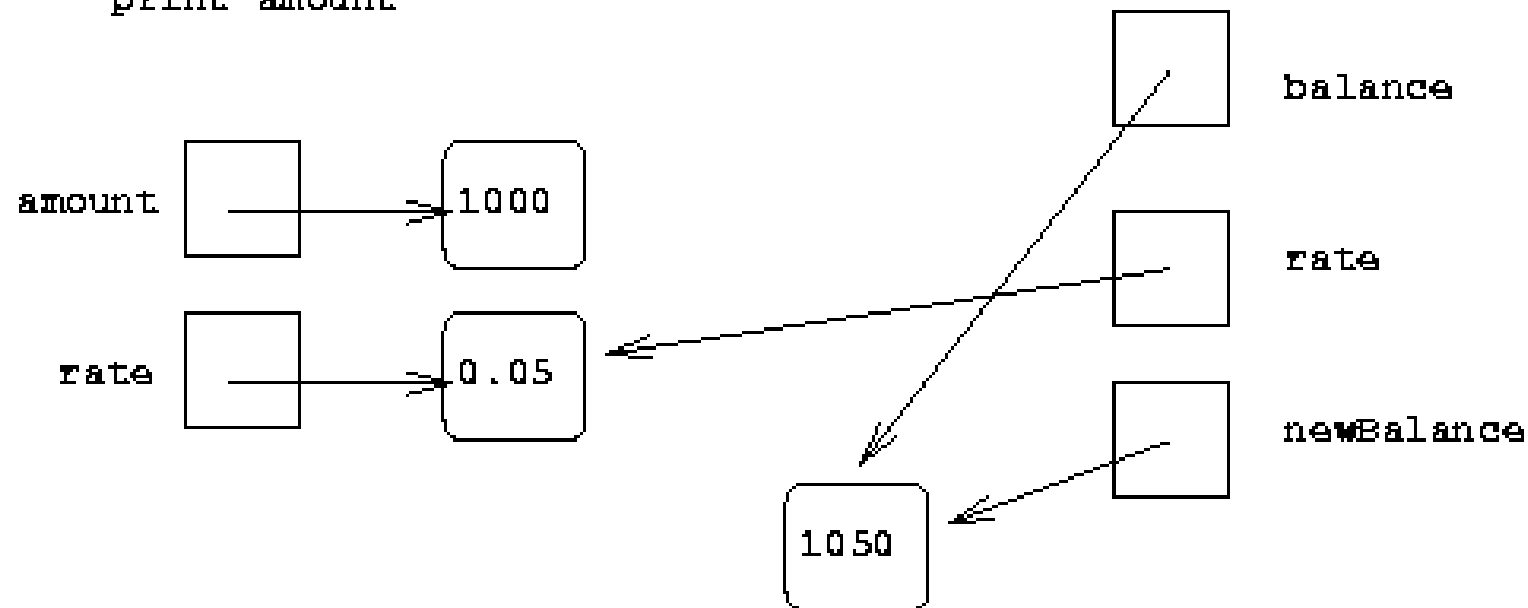


Behind the scene

```
def test():  
    amount = 1000  
    rate = 0.05  
    addInterest(amount, rate)  
    print amount
```

balance ← amount
rate ← rate

```
def addInterest(balance, rate):  
    newBalance = balance * (1 + rate)  
    balance = newBalance
```



Exercise 5.9



Will this work?

```
def addInterest(balance, rate):  
    balance = balance * (1 + rate)  
  
def test():  
    amount = 1000  
    rate = 0.05  
    addInterest(amount, rate)  
    print("My current balance is: ", amount)  
test()
```

For this case,

Same as before, when the arguments are passed to the parameters, both `balance` and `amount` reference to the same location where 1000 is stored.

However, when `balance` is assigned to a new value, due to the fact that integer is immutable, the new value is stored in a new location.

Therefore, `amount` will not be updated by the new value referenced by `balance`.

Exercise 5.10



Try the code on the next slide. Is the result expected?

Another example



```
def addInterest(balances, rate):  
    for i in range(len(balances)):  
        balances[i] = balances[i] * (1+rate)
```

```
def test():  
    amounts = [1000, 2200, 800, 360]  
    rate = 0.05  
    addInterest(amounts, 0.05)  
    print(amounts)
```

```
test()
```

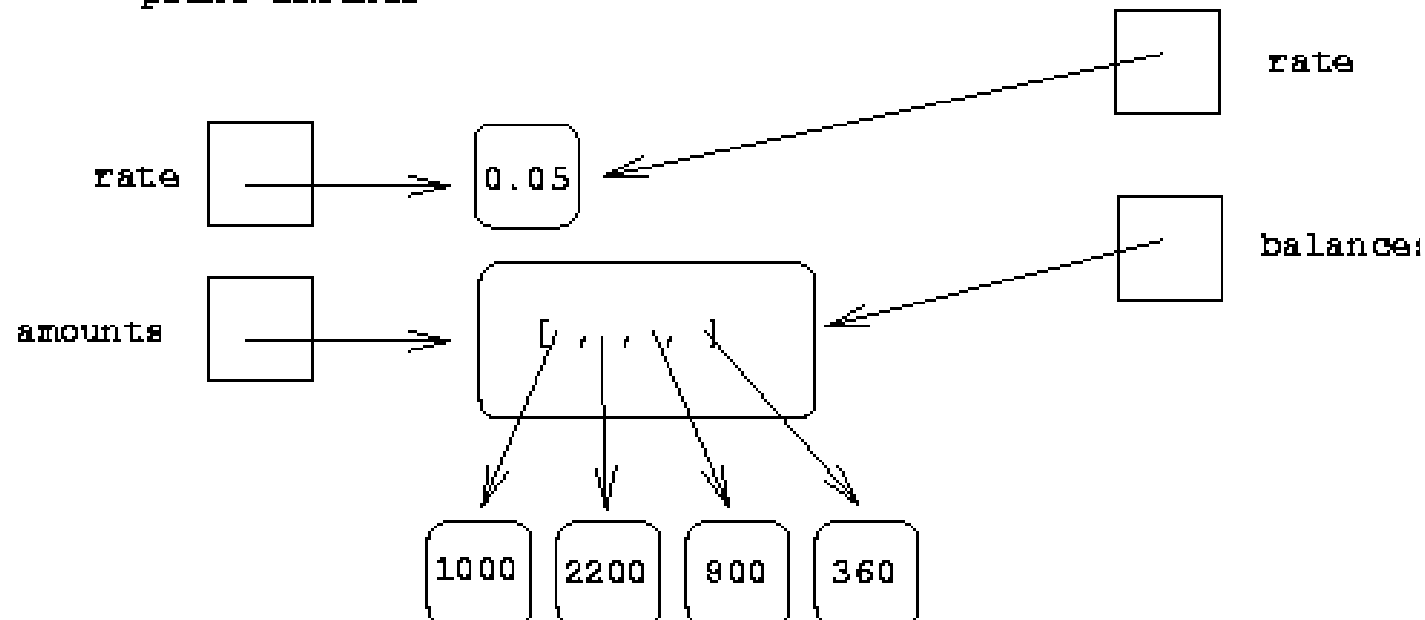
Behind the scene



```
def test():  
    amounts = [1000, 2150, 900, 3275]  
    rate = 0.05  
    addInterest(amounts, rate)  
    print amounts
```

↙

```
def addInterest(balances, rate):  
    for i in range(len(balances)):  
        balances[i] = balances[i] * (1+rate)
```

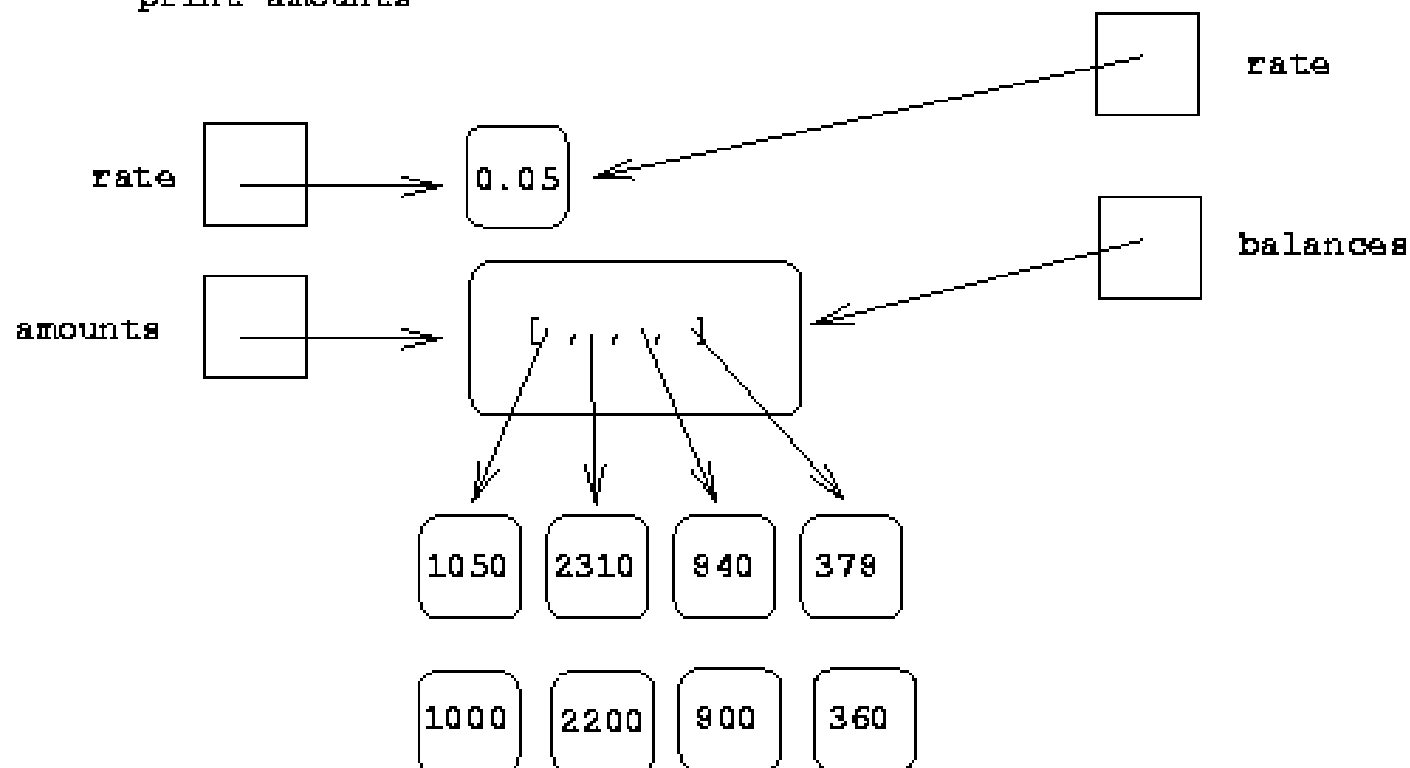


Behind the scene



```
def test():  
    amounts = [1000, 2150, 900, 3275]  
    rate = 0.05  
    addInterest(amounts, rate)  
    print amounts
```

```
def addInterest(balances, rate):  
    for i in range(len(balances)):  
        balances[i] = balances[i] * (1+rate)
```



For this case,



Unlike the previous cases, this time the amounts are stored in a list.

When the arguments are passed to the parameters, both `balance` and `amount` reference to the same location where the list is stored.

When `balance` is updated, since list is mutable, the values in the list will be updated.

A few points



Each value is given by multiple names (i.e., name aliasing)

The old values have not changed, but the new values were created and assigned into the list.

Will this work?

```
def changeString(string):  
    string[0] = "A"
```

```
name = "RockyChang"  
changeString(name)
```

Exercise 5.11



Try

```
def funList(aList):  
    aList.append(1)  
    aList = [2,3]
```

```
L1 = [0]  
funList(L1)  
print(L1)
```

Local scope and local variables

Scope: places in a program where the parameters and variables are referenced.

A **local variable** is a variable that is only accessible from within a given function. Such variables are said to have **local scope**.

Formal parameters also have local scope.

Local variables are automatically created (allocated memory) when a function is called, and destroyed (deallocated) when the function terminates.

Exercise 5.12



Visualize the following codes using Python tutor
(<http://www.pythontutor.com>)

```
n1, n2, n3 = 100, 200, 300
def avg(n1, n2, n3):
    print((n1 + n2 + n3)/3)

avg(10, 20, 30)
print(n1, n2, n3)
```

Global variables and global scope

A **global variable** is a variable that is defined outside of any function definition. Such variables are said to have **global scope**.

For example.

```
max_count = 1000    # a global variable
def funA(count):
    if count < max_count:
        ...
def funB(count):
    for i in range(max_count):
        ...
```

The use of global variables is generally considered to be **bad programming style**.

Exercise 5.13



Below is a well-known way to compute the value of e :

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + \frac{1}{1} + \frac{1}{1 \cdot 2} + \frac{1}{1 \cdot 2 \cdot 3} + \dots$$

Implement it using Python. Ask user for a maximum n and print out the value of each round. A sample output is on the next page. **Your program should consist of at least three functions.**



Enter a positive integer for approximating e: 10

The value of e is 2.718281828459045.

Round The approximated e

1	1.0
2	2.0
3	2.5
4	2.66666666666666665
5	2.7083333333333333
6	2.71666666666666663
7	2.7180555555555554
8	2.7182539682539684
9	2.71827876984127
10	2.7182815255731922

6. Dictionaries and sets

Objectives

To learn two Python data models: dictionary and set

To apply these two data models to solve problems, such as counting the frequency of words

Python dictionary

A dictionary is a mutable, associative data structure of variable length.

- The key can be of any immutable type.
- The values can be of any type and are unordered.

An example:

- `dicta = {'rocky': 'CHANG Kow Chuen', ('a', 'b', 'c'): [1, 2, 3], 1: 2}`
- `divmod(x,y)` returns a tuple.
- Get the value using its index, e.g., `dicta["rocky"]`.

Dictionary operators

Operation	Results
<code>dict()</code>	Creates a new, empty dictionary
<code>dict(s)</code>	Creates a new dictionary with key values and their associated values from sequence <code>s</code> , for example, <pre>fruit_prices = dict(fruit_data)</pre> where <code>fruit_data</code> is (possibly read from a file): <pre>[['apples', .66], ..., ['bananas', .49]]</pre>
<code>len(d)</code>	Length (num of key/value pairs) of dictionary <code>d</code> .
<code>d[key] = value</code>	Sets the associated value for <code>key</code> to <code>value</code> , used to either add a new key/value pair, or replace the value of an existing key/value pair.
<code>del d[key]</code>	Remove <code>key</code> and associated value from dictionary <code>d</code> .
<code>key in d</code>	True if key value <code>key</code> exists in dictionary <code>d</code> , otherwise returns <code>False</code> .

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Storing students' records



Problem: How to store student's data in a program after reading the data from a file?

A possible way is to store the data in a list.

However, a list can be indexed only by non-negative integers, but ours should be indexed by names.

- Map the names to a set of integers.
- Use the integers as indices for accessing the list.

A much better solution is to use a Python dictionary that maps the set of names (keys) directly to a set of values.

Exercise 6.1



Try

- `nicknames = dict()`
- `print(nicknames)`
- `nicknames = {"Rocky" : "CHANG Kow Chuen",
"Memory" : "CHIU Wai Han"}`
- `print(nicknames)`
- `nicknames["Dennis"] = "LIU Yan Wang"`
- `print(nicknames)`

Exercise 6.2



Try

- Add an integer-typed key and an integer-typed value.
- Add an integer-typed key and a list-typed value.
- Add a list-typed key and a list-typed value.

Exercise 6.3



Create a dictionary of dictionary, such as

```
dicta = {"a":1, "b":{"aa":11, "bb":22}}.
```

How do you get the values in the inner dictionary (i.e., 11 and 22)?

Exercise 6.4



Create a dictionary dicta and try

- `len(dicta)`
- Use the keyword "in" to check whether an object is a key in dicta.
- Use a for loop to print out all the keys in dicta.
- Use a for loop to print out all the keys and the corresponding values on a new line.

Dictionary methods



Python provides a number of other methods that iterate through the elements in a dictionary.

- `items()`: all the key-values pairs as a list of tuples
- `keys()`: all the keys as a list
- `values()`: all the values as a list

The objects returned by `dict.keys()`, `dict.values()` and `dict.items()` are *view objects*.

- They provide a dynamic view on the dictionary's entries, which means that when the dictionary changes, the view reflects these changes.
- They are iterable, therefore can be used in a loop.

Exercise 6.5



Create a dictionary and use the three methods and a loop to print out their keys only, values only, and key-value.

Exercise 6.6



Write a program that will count the number of occurrences of each word in a file and store them in a dictionary. Print the key-value on a new line. Assume that the file does not contain any punctuations, symbols, and numbers.

Exercise 6.7



In this exercise, we would like to print out the statistics in a sorted order of the words. In order to do this, you need to create a list of tuples first for the key-value in the dictionary and then apply `sorted()`.

Exercise 6.8



Given a dictionary d and a key k , it is easy to find the corresponding value $v = d[k]$. This operation is called a lookup. Write a function for reverse lookup. That is, given d and a value v , the function will return the first key that maps to v .

Exercise 6.9



Modify `reverse_lookup()` in exercise 6.8 so that it builds and returns a list of *all* keys that map to v , or an empty list if there are none.

Sets in Python

A set is collection of non-mutable objects which are elements of the set.

Set is mutable.

The elements could be of any types.

There is no order in a set.

There are no duplicate elements in a set.

`{}` is an empty set.

Set operators

Set operator		Set A = {1,2,3}	Set B = {3,4,5,6}	
membership		<code>1 in A</code>	True	<i>True if 1 is a member of set</i>
add		<code>A.add(4)</code>	{1,2,3,4}	<i>Adds new member to set</i>
remove		<code>A.remove(2)</code>	{1,3}	<i>Removes member from set</i>
union		<code>A B</code>	{1,2,3,4,5,6}	<i>Set of elements in either set A or set B</i>
intersection		<code>A & B</code>	{3}	<i>Set of elements in both set A and set B</i>
difference		<code>A - B</code>	{1,2}	<i>Set of elements in set A, but not set B</i>
symmetric difference		<code>A ^ B</code>	{1,2,4,5,6}	<i>Set of elements in set A or set B, but not both</i>
size		<code>len(A)</code>	3	<i>Number of elements in set (general sequence operation)</i>

Source: Charles Dierbach. 2013. Introduction to Computer Science Using Python. Wiley.

Exercise 6.10



- Create a set using `set()` and `{}` with duplicate elements.
- Create an empty set.
- Create a set of a list as an element.
- Use `len()` to get the size.
- Use the keyword "in" to test whether an object is an element of a set.
- Use a for loop to print all the elements of a set.

Exercise 6.11



Try all four set operations.

You can also use four methods for sets: intersection, union, difference, symmetric_difference. For example, `set_A.union(set_B)`.



7. Modules

Objectives

Explain the specification of modules.

Become familiar with the use of docstrings in Python.

Explain the use of modules and namespaces in Python.

Describe and use the different forms of module import in Python.

Explain the local, global, and built-in namespaces and the scope resolution.

Where do the modules come from?

- The term “module” refers to the design and/or implementation of specific functionality to be incorporated into a program.

- Modular-II

```
MODULE Hello;  
FROM StextIO IMPORT WriteString;  
BEGIN  
    WriteString("Hello World!");  
END Hello.
```

A Python module is a file containing Python definitions and statements.

- By convention, modules are named using all lower case letters and optional underscore characters.

Exercise 7.1

- Import a module.
- Use `help(module_name)` or `dir(module_name)` to find out the functions available in that module.
- Use `print(module_name)` to find out what the module does.
- Use `print(function_name.__doc__)` to print out what the function does.

Interface, client and docstring

A module's *interface* is a specification of what it provides and how it is to be used.

Any program code making use of a given module is called a *client* of the module.

A *docstring* is a string literal denoted by triple quotes used in Python for providing the specification of certain program elements.

Exercise 7.2

Try

- `import graphics module`
- `dir (graphics)`
- `help (graphics)`
- `print (graphics.__doc__)`

Documentation String (docstring)

[http://legacy.python.org/dev/peps/pep-0008/#documentation-](http://legacy.python.org/dev/peps/pep-0008/#documentation-strings)

A *docstring* is a string literal denoted by triple quotes given as the first line of certain program elements.

These additional lines must be indented at the same level.

The docstring for a module is simply the first block of quoted text to appear in the module.

Write docstrings for all public modules, functions, classes, and methods.

Convention:

```
"""Return a foobang
```

```
Optional plotz says to frobnicate the bizbaz  
first. """
```

Exercise 7.3



Write docstrings for your previous assignment.

Exercise 7.4



- Run `graphics.py` as a main module.
- Run `graphics.py` as an imported module. For example, draw some polygons.
- What is the difference between the two?

Running a module as main or imported module

Before executing the code, the Python interpreter will define a few special variables, e.g., `__name__`.

If the python interpreter is running that module (the source file) as the main program, it sets the special `__name__` variable to have a value `"__main__"`.

If this file is being imported from another module, `__name__` will be set to the module's name.

`test()` in `graphics.py` will be executed only when the module is run as main:

```
if __name__ == "__main__":  
    test()
```

See https://docs.python.org/3/library/__main__.html

Exercise 7.5



- Create a simple module `a` (e.g., just printing out a message) and put it in your folder for modules (not in the `Lib` folder under `Python-3.4.1`)
- Create another simple module `b` that imports module `a`.
- Run module `b`.

Searching for imported modules

Each imported module of a Python program needs to be located and loaded into memory.

Python first searches for modules in the current directory. If the module is not found, it searches the directories specified in the `PYTHONPATH` environment variable.

If the module is still not found, a Python installation-specific path is searched (e.g., `C:\Python32\Lib`).

If the program still does not find the module, an `ImportError` exception is reported.

The main and imported modules

The main module may *import* any number of other modules (and each of those modules import other modules, etc.).

Main modules are not meant to be imported into other modules.

The statements of imported modules are executed only once, the first time that the module is imported.

- When a module is loaded, a compiled version of the module with file extension `.pyc` is automatically produced.

Imported modules:

- Standard library (usually written in C) or
- Other modules (usually written in Python)

Exercise 7.6



- Try the code on the next page. What will be printed out?
- Next, comment `x=1` inside `funA()` and run it again.
- Now, add `x = 10` before `x = funA()` and run it again.



```
# A module
```

```
def funA():  
    # x is a local variable  
    x = 1  
    print(x)  
    return x
```

```
def funB():  
    # x is a local variable  
    x = 2  
    print(x)  
    return x
```

```
# x is a global variable  
x = funA()  
x = funB()  
x = 0  
print(x)
```

The local assignment rule

If anywhere in a function an assignment is made, then that assignment is assumed to create a name *only in the presently active namespace*.

If, within a function, a variable is declared to be a global variable using the `global` statement, then Python will not create a local name in the namespace for that variable.

Modules and Namespaces

A *namespace* is a container that provides a named context for a set of identifiers (for variables, functions, and constants).

A *name clash* is when two otherwise distinct entities with the same name become part of the same scope.

In Python, each module has its own namespace.

- This includes the names of all items in the module.

The *fully qualified* name of each identifier in a module is of the form *modulename.identifier*.

`math.sqrt()` in a (e.g., `math`) module is used when the module is imported.

Exercise 7.8



Use the interactive mode for this exercise.

1. Restart the shell.
2. Type `dir()`.
3. Run the module in Exercise 7.6.
4. Type `dir()`.
5. Repeat steps 1-4 for `graphics.py`.

A module's/global namespace



In Python, the main module of any program is the first (“top-level”) module executed.

When working interactively in the Python shell, the Python interpreter functions as the main module, containing the *global namespace*.

- The namespace is reset every time the interpreter is started.

Six names are always there: `'__builtins__'`,
`'__doc__'`, `'__loader__'`, `'__name__'`,
`'__package__'`, `'__spec__'`

Exercise 7.9



In the shell used for the last exercise, import another module and then find out the updated namespace by `dir()`. Consider the following four cases:

- `import math`
- `from math import sqrt, log`
- `from math import sqrt as new_sqrt`
- `from math import *`

Importing modules



When using `import module_name`, the namespace of the imported module becomes *available to*, but not *part of*, the importing module.

- The imported identifiers must be referenced fully qualified.

When using `from module_name import something`, the imported module's namespace *becomes part of* the importing module's namespace.

- The imported identifiers are referenced without being fully qualified.

From-import



- Three possible from-imports:
 - `from module_name import func1, func2`
 - `from module_name import func1 as new_func1`
 - `from module_name import *`
- For `import *`,
 - All of the identifiers are imported, except for those that begin with two underscore characters, which are meant to be *private* in the module.
- In Python, all the variables in a module are “public,” with the convention that variables beginning with two underscores are intended to be private.

Exercise 7.10



Try

```
from math import *
```

Define a new function:

```
def ceil(x):  
    if x - int(x) >= 0.5:  
        return int(x)+1  
    else:  
        return int(x)
```

Execute `ceil()` to see which `ceil()` is used.

Execute `math.ceil()`.

Local, Global, and Built-in Namespace



During a Python program's execution, there are as many as three namespaces:

- The *built-in namespace* contains the names of all the built-in functions, constants, and so on, in Python.
- The *global namespace* contains the identifiers of the currently executing module.
- The *local namespace* is the namespace of the currently executing function (if any).

Exercise 7.11



Run the module in the next page.

Try to understand what the outputs tell you.



```
import math
globalX = 100

def getlocalnamespace(p1=123, p2="hihi"):
    localX = 10
    print("\n=== local namespace ===")
    for key, val in locals().items():
        print("key: {0:10} object: {1:10}".format(key, str(val)))

def getglobalnamespace():
    print("\n=== global namespace ===")
    for key, val in globals().items():
        print("key: {0:20} object: {1:20}".format(key, str(val)))

getlocalnamespace()
getglobalnamespace()
```

locals () and globals () but in functions



`locals ()` : Update and return a dictionary representing the current local symbol table.

`globals ()` : Return a dictionary representing the current global symbol table.

Exercise 7.12



Run the module in the next page.
Try to understand what the outputs tell
you.



```
def getbuiltins():
    builtin_dict = __builtins__.__dict__

    print("\nBuiltin dictionary has {} entries\n".format(len(builtin_dict)))
    for key, val in builtin_dict.items():
        print("key: {0:20} object: {1:20}".format(key, str(val)))

getbuiltins()
```

The builtins module



When the Python interpreter starts up, it loads two default modules without requiring an import:

`__main__` and `__builtin__`.

The `__builtin__` module provides the built-in functions and built-in constants.

More details on this module:

<https://docs.python.org/3/library/builtins.html>



8. Files

Objectives



To understand basic file processing concepts and techniques for reading and writing text files in Python.

To be able to understand and write programs that process textual information.

File (object)



A *file* is a sequence of data that is stored in secondary memory (disk drive).

Two types of files: *text file* and *binary file*

- A text file contains characters, structured as lines of text.
- A binary file is a file formatted in a way that only a computer program can read.

A text file usually contains more than one line of text. Lines of text are separated with a special character, the *newline* character.

Exercise 8.1



- Open notepad++ (you may need to install it yourself in J drive)
- Find a .py file and use notepad++ to open it.
- Under “View/Show Symbols”, choose “Show All Characters” to see the newline and tab characters.

How to end a line?

(<http://en.wikipedia.org/wiki/Newline>)



LF: Multics, Unix and Unix-like systems (GNU/Linux, OS X, FreeBSD, AIX, Xenix, etc.), BeOS, Amiga, RISC OS and others.

CR: Commodore 8-bit machines, Acorn BBC, ZX Spectrum, TRS-80, Apple II family, Mac OS up to version 9 and OS-9

CR+LF: Microsoft Windows, DEC TOPS-10, RT-11 and most other early non-Unix and non-IBM OSes, CP/M, MP/M, DOS (MS-DOS, PC DOS, etc.), Atari TOS, OS/2, Symbian OS, Palm OS, Amstrad CPC

Exercise 8.2



- Install and open binaryviewer in J drive.
- Open the same .py file used in the last exercise.
- Compare the binary representation on the left and the text on the right.

File Processing



Open a file in a secondary storage.

Read / write the file.

Save the file if write.

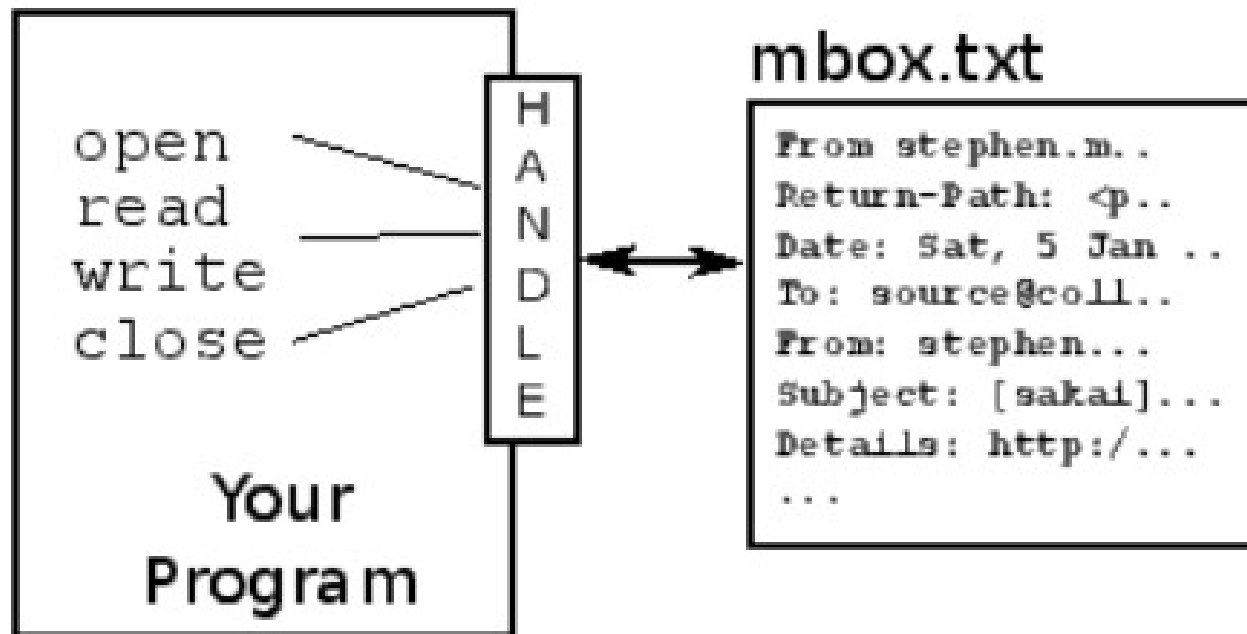
Close the file.

Opening a file in a secondary stor



If successful, a “file handler” will be returned.

- `infile = open("mbox.txt", "r")`
- `infile = open("mbox.txt", "w")`



Reading/writing and saving a fi



Load (part of) the file into the main memory.

Read the file from the memory.

Write the file to the memory.

Saving will write the file in the memory to a secondary storage.

Exercise 8.3



Try

```
infile = open("a_text_file_of_your_choice", "r")  
data = infile.read()  
print(data)
```

(You may need `os.getcwd()` and `os.chdir()` to find your files.)

Exercise 8.4



Try

```
infile = open("a_text_file_of_your_choice", "r")
for i in range(10):
    line = infile.readline()
    print(line[:-1])
```

- How do you print the lines in a single line?

Exercise 8.5



Try

```
infile = open("a_text_file_of_your_choice", "r")
for line in infile.readlines():
    print(line)
```

- Does the output look the same as the file's?

Three file read methods



`<filevar>.read()` – returns the entire remaining contents of the file as a single (possibly large, multi-line) string.

`<filevar>.readline()` – returns the next line of the file. This is all text up to *and including* the next newline character.

`<filevar>.readlines()` – returns a list of the remaining lines in the file. Each list item is a single line including the newline characters.

Exercise 8.6



Try

```
input_file =  
open("an_existing_py_file", "r")  
output_file = open("clone.py", "w")  
content = input_file.read()  
output_file.write(content)  
output_file.close()
```

Find the new clone.py file.

Exercise 8.7



Try

```
input_file = open("an_existing_py_file", "r")
output_file = open("another_existing_py_file", "a")
content = input_file.read()
output_file.write(content)
output_file.close()
```

Open the output file.

Write and append methods



Opening a file for writing prepares the file to receive data.

If you open an existing file for writing, you wipe out the file's contents. If the named file does not exist, a new one is created.

It is important to close a file that is written to, otherwise the tail end of the file may not be written to the file.

Exercise 8.8



- Download `userfile.py` from mcsp.wartburg.edu/zelle/python/ppics2/code/chapter05/userfile.py.
- Prepare an input file and use `userfile.py` to generate batch usernames.

Exercise 8.9



- **Modify** `userfile.py` by replacing `print(uname, file=outfile)` with `write()`. Do you observe any difference in the output file?
- How do you slightly modify the codes to generate the same outputs?

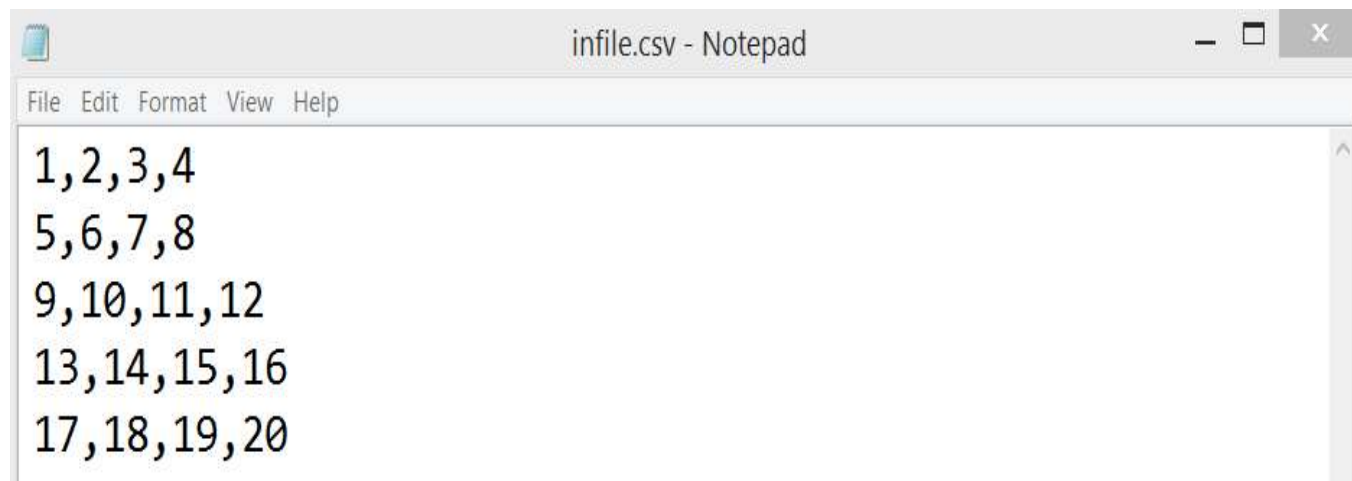
Exercise 8.10



Please find it on the next few slides.



Write a Python program that has two parts. The first is to ask users to input a starting number and an ending number, and the number of numbers (say *co*) printed on each row and the number of spaces for printing each number. The program will then write the numbers to a file, and the numbers are separated by "," except for the last number on a line. After writing *co* numbers, a new line will be started. A sample of the file:



```
infile.csv - Notepad
File Edit Format View Help
1,2,3,4
5,6,7,8
9,10,11,12
13,14,15,16
17,18,19,20
```



In the second part, the program opens the file and prints them to the screen using the number of spaces specified by the user. Each row is filled up from left to right, and a new row will be used only after a row is filled up. Each number is printed on right-justified format. Three sample outputs:



```
>>> ===== RESTART =====
```

```
>>>
```

```
Please enter a starting number and an ending number: 1,20
```

```
Please enter the number of numbers printed on a row
```

```
and the number of spaces per item: 4,4
```

```
 1   2   3   4
 5   6   7   8
 9  10  11  12
13  14  15  16
17  18  19  20
```

```
>>> ===== RESTART =====
```

```
>>>
```

```
Please enter a starting number and an ending number: -20,20
```

```
Please enter the number of numbers printed on a row
```

```
and the number of spaces per item: 10, 5
```

```
-20 -19 -18 -17 -16 -15 -14 -13 -12 -11
-10  -9  -8  -7  -6  -5  -4  -3  -2  -1
 0   1   2   3   4   5   6   7   8   9
10  11  12  13  14  15  16  17  18  19
20
```

```
>>> ===== RESTART =====
```

```
>>>
```

```
Please enter a starting number and an ending number: 0,0
```

```
Please enter the number of numbers printed on a row
```

```
and the number of spaces per item: 5,5
```

```
0
```

```
>>>
```

9. Classes

Fundamental concepts of OOP in Python

The four major principles of object orientation are:

- Encapsulation
- Data Abstraction
- Inheritance
- Polymorphism

What is an Object..?



- Objects are the basic run-time entities in an object-oriented system.
- They may represent a person, a place, a bank account, a table of data or any item that the program must handle.
- When a program is executed the objects interact by sending messages to one another.
- Objects have two components:
 - Data (i.e., attributes)
 - Behaviors (i.e., methods)

What is a Class..?



- A class is a special data type which defines how to build a certain kind of object.
- The class also stores some data items that are shared by all the instances of this class
- Instances are objects that are created which follow the definition given inside of the class
- Python doesn't use separate class interface definitions as in some languages
- You just define the class and then use it

Methods in Classes

- Define a method in a class by including function definitions within the scope of the class block
- There must be a special first argument **self** in all of method definitions which gets bound to the calling instance
- There is usually a special method called **__init__** in most classes

A Simple Class def: Student

```
class student:
    """A class representing a student """
    def __init__(self, n, a):
        self.full_name = n
        self.age = a
    def get_age(self):    #Method
        return self.age
```

- Define class:
 - Class name, begin with capital letter, by convention
 - object: class based on (Python built-in type)
- Define a method
 - Like defining a function
 - Must **have a special first parameter**, self, which provides way for a method to refer to object itself

Instantiating Objects with ‘__init__’

- `__init__` is the default constructor
- `__init__` serves as a constructor for the class. Usually does some initialization work
- An `__init__` method can take any number of arguments
- However, the first argument `self` in the definition of `__init__` is special

Self

- The first argument of every method is a reference to the current instance of the class
- By convention, we name this argument **self**
- In `__init__`, `self` refers to the object currently being created; so, in other class methods, it refers to the instance whose method was called
- Similar to the keyword **this** in Java or C++
- But Python uses **self** more often than Java uses **this**
- You **do not** give a value for this parameter(`self`) when you call the method, Python will provide it.

Continue...

...Continue

- Although you must specify self explicitly when defining the method, you don't include it when calling the method.
- Python passes it for you automatically

Defining a method:

Calling a method:

(this code inside a class definition.)

```
def get_age(self, num):  
    self.age = num
```

```
>>> x.get_age(23)
```

Deleting instances: No Need to “free”

- When you are done with an object, you don't have to delete or free it explicitly.
- Python has automatic garbage collection.
- Python will automatically detect when all of the references to a piece of memory have gone out of scope. Automatically frees that memory.
- Generally works well, few memory leaks
- There's also no “destructor” method for classes

Syntax for accessing attributes and methods

```
>>> f = student("Python", 14)
```

```
>>> f.full_name # Access attribute  
"Python"
```

```
>>> f.get_age() # Access a method  
14
```

Encapsulation

- Important advantage of OOP consists in the encapsulation of data. We can say that object-oriented programming relies heavily on encapsulation.
- The terms encapsulation and abstraction (also data hiding) are often used as synonyms. They are nearly synonymous, i.e. abstraction is achieved through encapsulation.
- Data hiding and encapsulation are the same concept, so it's correct to use them as synonyms
- Generally speaking encapsulation is the mechanism for restricting the access to some of an object's components, this means, that the internal representation of an object can't be seen from outside of the object's definition.

-
-
- Access to this data is typically only achieved through special methods: **Getters** and **Setters**
 - By using solely get() and set() methods, we can make sure that the internal data cannot be accidentally set into an inconsistent or invalid state.
 - C++, Java, and C# rely on the public, private, and protected keywords in order to implement variable scoping and encapsulation
 - It's nearly always possible to circumvent this protection mechanism

Public, Protected and Private Data

- If an identifier doesn't start with an underscore character "_" it can be accessed from outside, i.e. the value can be read and changed
- Data can be protected by making members private or protected. Instance variable names starting with two underscore characters cannot be accessed from outside of the class.
- At least not directly, but they can be accessed through private name mangling.
- That means, private data `__A` can be accessed by the following name construct: *`instance_name._classname__A`*

-
- If an identifier is only preceded by one underscore character, it is a protected member.
 - Protected members can be accessed like public members from outside of class

Example:

```
class Encapsulation(object):  
    def __init__(self, a, b, c):  
        self.public = a  
        self._protected = b  
        self.__private = c
```

The following interactive sessions shows the behavior of public,
protected and private members:

```
>>> x = Encapsulation(11,13,17)
```

```
>>> x.public
```

```
11
```

```
>>> x._protected
```

```
13
```

```
>>> x._protected = 23
```

```
>>> x._protected
```

```
23
```

```
>>> x.__private
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
AttributeError: 'Encapsulation' object has no attribute '__private'
```

```
>>>
```


The following table shows the different behavior Public, Protected and Private Data

Name	Notation	Behavior
name	Public	Can be accessed from inside and outside
_name	Protected	Like a public member, but they shouldn't be directly accessed from outside
__name	Private	Can't be seen and accessed from outside

Inheritance

- Inheritance is a powerful feature in object oriented programming
- It refers to defining a new class with little or no modification to an existing class.
- The new class is called derived (or child) class and the one from which it inherits is called the base (or parent) class.
- Derived class inherits features from the base class, adding new features to it.
- This results into re-usability of code.

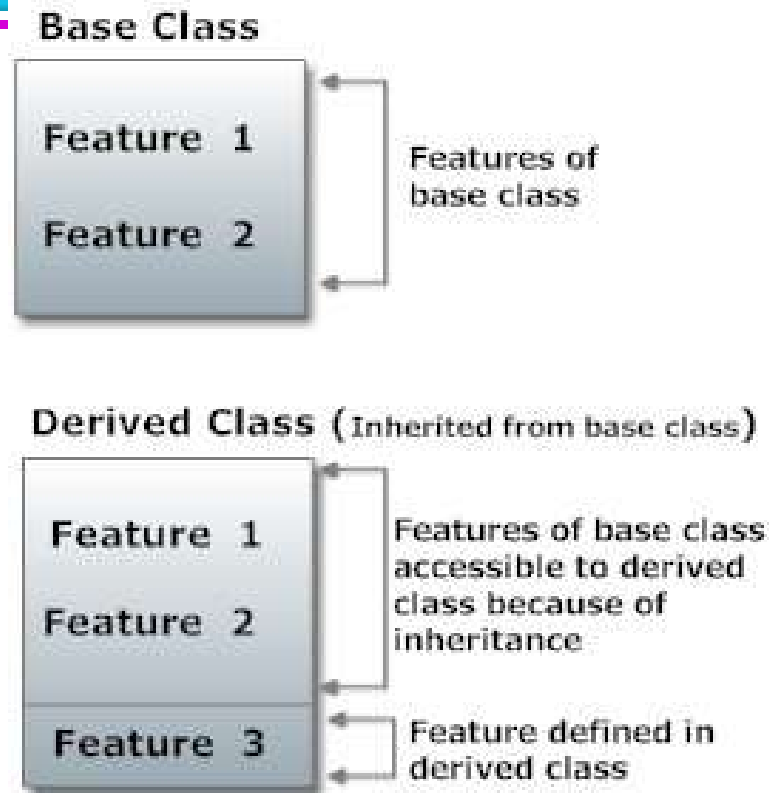
Syntax:

```
class Baseclass(Object):
```

```
    body_of_base_class
```

```
class DerivedClass(BaseClass):
```

```
    body_of_derived_clas
```



While designing a inheritance concept, following key pointes keep it in mind

- A sub type never implements less functionality than the super type
- Inheritance should never be more than two levels deep
- We use inheritance when we want to avoid redundant code.

Two built-in functions *isinstance()* and *issubclass()* are used to check inheritances.

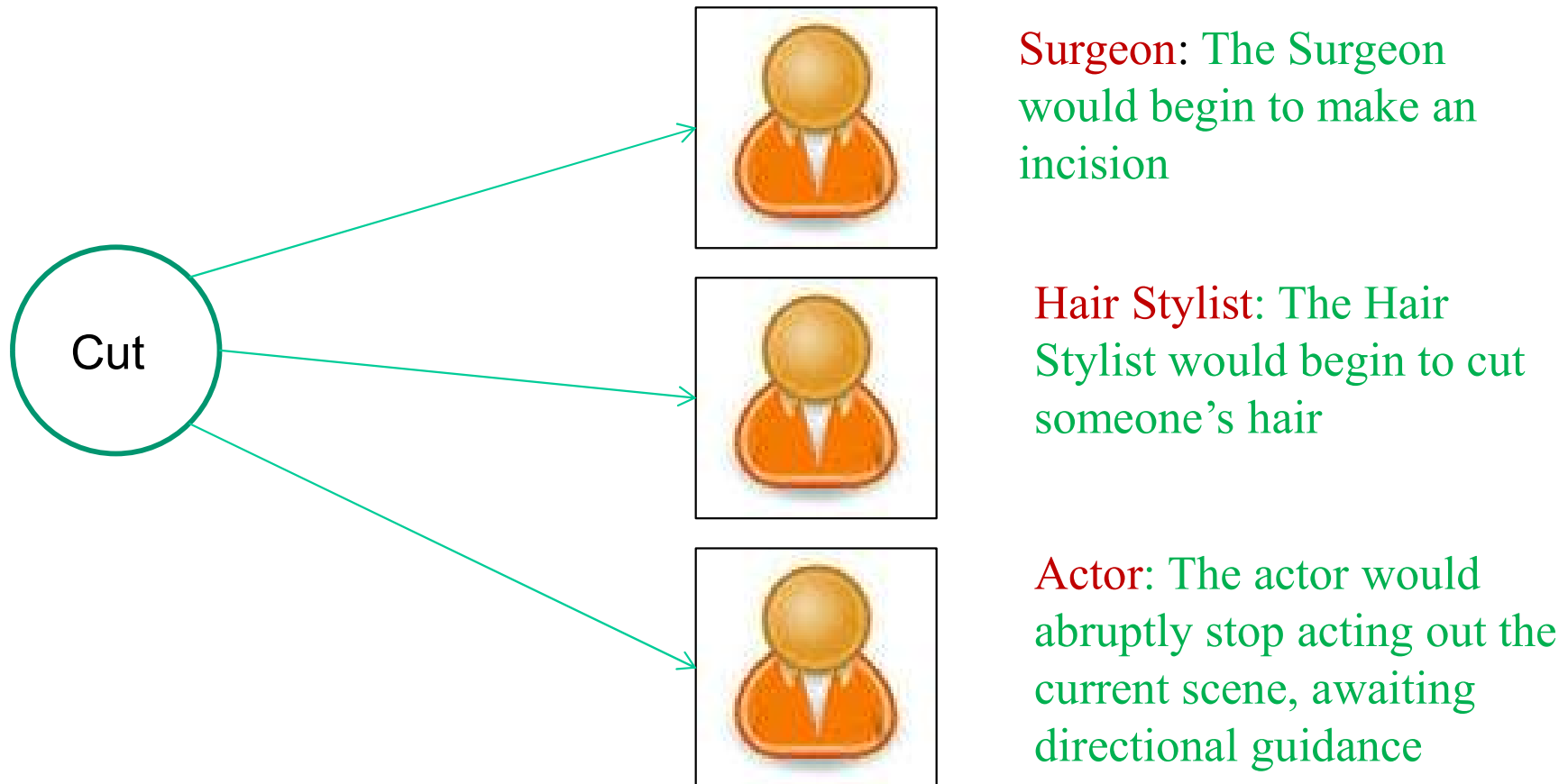
- Function *isinstance()* returns True if the object is an instance of the class or other classes derived from it.
- Each and every class in Python inherits from the base class object.

Polymorphism

- Polymorphism in Latin word which made up of '*ploy*' means many and '*morphs*' means forms
- From the Greek , Polymorphism means *many(poly) shapes (morph)*
- This is something similar to a word having several different meanings depending on the context
- Generally speaking, polymorphism means that a method or function is able to cope with different types of input.

A simple word 'Cut' can have different meaning depending where it is used

If any body says “Cut” to these people



In OOP , Polymorphism is the characteristic of being able to assign a different meaning to a particular symbol or operator in different contexts specifically to allow an entity such as a variable, a function or an object to have more than one form.

There are two kinds of Polymorphism

Overloading :

Two or more methods with different signatures

Overriding:

Replacing an inherited method with another having the same signature

Operator Overloading

- Python operators work for built-in classes.
- But same operator behaves differently with different types. For example, the `+` operator will, perform arithmetic addition on two numbers, merge two lists and concatenate two strings. This feature in Python, that allows same operator to have different meaning according to the context is called operator overloading
- One final thing to mention about operator overloading is that you can make your custom methods do whatever you want. However, common practice is to follow the structure of the built-in methods.

Explanation for Operator Overloading Sample Program

What actually happens is that, when you do `p1 - p2`, Python will call `p1.__sub__(p2)` which in turn is `Point.__sub__(p1,p2)`. Similarly, we can overload other operators as well. The special function that we need to implement is tabulated below.

Operator	Expression	Internally
Addition	<code>p1 + p2</code>	<code>p1.__add__(p2)</code>
Subtraction	<code>p1 - p2</code>	<code>p1.__sub__(p2)</code>
Multiplication	<code>p1 * p2</code>	<code>p1.__mul__(p2)</code>
Power	<code>p1 ** p2</code>	<code>p1.__pow__(p2)</code>
Division	<code>p1 / p2</code>	<code>p1.__truediv__(p2)</code>